

Determining An Internet Address At Startup (RARP)

6.1 Introduction

We now know that physical network addresses are both low-level and hardware dependent, and we understand that each machine using TCP/IP is assigned one or more 32-bit IP addresses that are independent of the machine's hardware addresses. Application programs always use the IP address when specifying a destination. Because hosts and routers must use a physical address to transmit a datagram across an underlying hardware network; they rely on address resolution schemes like ARP to map between an IP address and an equivalent hardware address.

Usually, a computer's IP address is kept on its secondary storage, where the operating system finds it at startup. The question arises, "How does a machine without a permanently attached disk determine its IP address?" The problem is critical for workstations that store files on a remote server or for small embedded systems because such machines need an IP address before they can use standard TCP/IP file transfer protocols to obtain their initial boot image. This chapter explores the question of how to obtain an IP address, and describes a low-level protocol that such machines can use before they boot from a remote file server. Chapter 23 extends the discussion of bootstrapping, and considers popular alternatives to the protocol presented here.

Because an operating system image that has a specific IP address bound into the code cannot be used on multiple computers, designers usually try to avoid compiling a machine's IP address in the operating system code or support software. In particular, the bootstrap code often found in Read Only Memory (ROM) is usually built so the same image can run on many machines. When such code starts execution, it uses the network to contact a server and obtain the computer's IP address.

The bootstrap procedure sounds paradoxical: a machine communicates with a remote server to obtain an address needed for communication. The paradox is only imagined, however, because the machine *does* know how to communicate. It can use its physical address to communicate over a single network. Thus, the machine must resort to physical network addressing temporarily in the same way that operating systems use physical memory addressing to set up page tables for virtual addressing. Once a machine knows its IP address, it can communicate across an internet.

The idea behind finding an IP address is simple: a machine that needs to know its address sends a request to a *server*[†] on another machine, and waits until the server sends a response. We assume the server has access to a disk where it keeps a database of internet addresses. In the request, the machine that needs to know its internet address must uniquely identify itself, so the server can look up the correct internet address and send a reply. Both the machine that issues the request and the server that responds use physical network addresses during their brief communication. How does the requester know the physical address of a server? Usually, it does not — it simply broadcasts the request to all machines on the local network. One or more servers respond.

Whenever a machine broadcasts a request for an address, it must uniquely identify itself. What information can be included in its request that will uniquely identify the machine? Any unique hardware identification suffices (e.g., the CPU serial number). However, the identification should be something that an executing program can obtain easily. Unfortunately, the length or format of CPU-specific information may vary among processor models, and we would like to devise a server that accepts requests from all machines on the physical network using a single format. Furthermore, engineers who design bootstrap code attempt to create a single software image that can execute on an arbitrary processor, and each processor model may have a slightly different set of instructions for obtaining a serial number.

6.2 Reverse Address Resolution Protocol (RARP)

The designers of TCP/IP protocols realized that there is another piece of uniquely identifying information readily available, namely, the machine's physical network address. Using the physical address as a unique identification has two advantages. Because a host obtains its physical addresses from the network interface hardware, such addresses are always available and do not have to be bound into the bootstrap code. Because the identifying information depends on the network and not on the CPU vendor or model, all machines on a given network will supply uniform, unique identifiers. Thus, the problem becomes the reverse of address resolution: given a physical network address, devise a scheme that will allow a server to map it into an internet address.

The TCP/IP protocol that allows a computer to obtain its IP address from a server is known as the *Reverse Address Resolution Protocol (RARP)*. RARP is adapted from the ARP protocol of the previous chapter and uses the same message format shown in Figure 5.3. In practice, the RARP message sent to request an internet address is a little more general than what we have outlined above: it allows a machine to request the IP

[†]Chapter 21 discusses servers in detail.

address of a third party as easily as its own. It also allows for multiple physical network types.

Like an ARP message, a RARP message is sent from one machine to another encapsulated in the data portion of a network frame. For example, an Ethernet frame carrying a RARP request has the usual preamble, Ethernet source and destination addresses, and packet type fields in front of the frame. The frame type contains the value 8035_{16} to identify the contents of the frame as a RARP message. The data portion of the frame contains the 28-octet RARP message.

Figure 6.1 illustrates how a host uses RARP. The sender broadcasts a RARP request that specifies itself as both the sender and target machine, and supplies its physical network address in the target hardware address field. All computers on the network receive the request, but only those authorized to supply the RARP service process the request and send a reply; such computers are known informally as *RARP servers*. For RARP to succeed, the network must contain at least one RARP server.

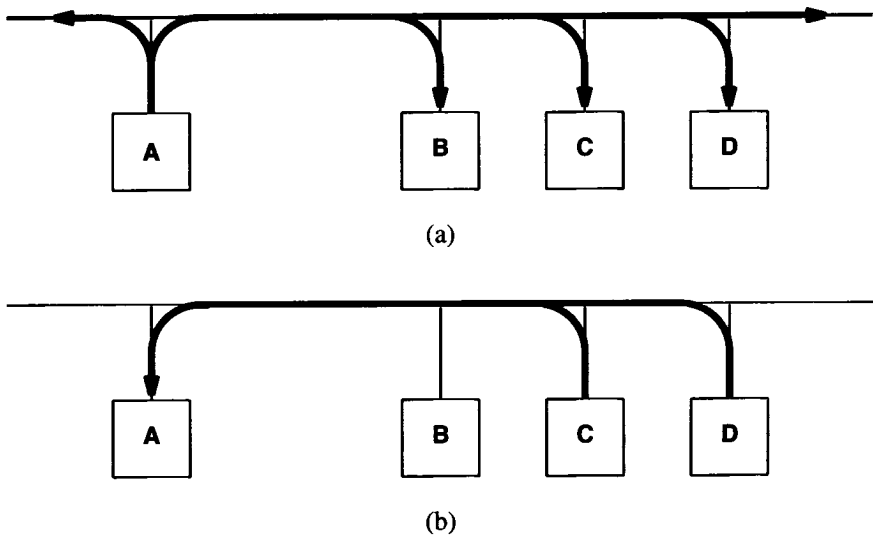


Figure 6.1 Example exchange using the RARP protocol. (a) Machine A broadcasts a RARP request specifying itself as a target, and (b) those machines authorized to supply the RARP service (C and D) reply directly to A.

Servers answer requests by filling in the target protocol address field, changing the message type from *request* to *reply*, and sending the reply back directly to the machine making the request. The original machine receives replies from all RARP servers, even though only the first is needed.

Keep in mind that all communication between the computer seeking its IP address and the server supplying it must be carried out using only the physical network. Furthermore, the protocol allows a host to ask about an arbitrary target. Thus, the sender supplies its hardware address separate from the target hardware address, and the server is careful to send the reply to the sender's hardware address. On an Ethernet, having a field for the sender's hardware address may seem redundant because the information is also contained in the Ethernet frame header. However, not all Ethernet hardware provides the operating system with access to the physical frame header.

6.3 Timing RARP Transactions

Like any communication on a best-effort delivery network, RARP requests and responses are susceptible to loss (including discard by the network interface if the CRC indicates that the frame was corrupted). Because RARP uses the physical network directly, no other protocol software will time the response or retransmit the request; RARP software must handle these tasks. In general, RARP is used only on local area networks like the Ethernet, where the probability of failure is low. If a network has only one RARP server, however, that machine may not be able to handle the load, so packets may be dropped.

Some computers that rely on RARP to boot, choose to retry indefinitely until they receive a response. Other implementations announce failure after only a few tries to avoid flooding the network with unnecessary broadcast traffic (e.g., in case the server is unavailable). On an Ethernet, network failure is less likely than server overload. Making RARP software retransmit quickly may have the unwanted effect of flooding a congested server with more traffic. Using a large delay ensures that servers have ample time to satisfy the request and return an answer.

6.4 Primary And Backup RARP Servers

The chief advantage of having several computers function as RARP servers is that it makes the system more reliable. If one server is down or too heavily loaded to respond, another answers the request. Thus, it is highly likely that the service will be available. The chief disadvantage of using many servers is that when a machine broadcasts a RARP request, the network becomes overloaded because all servers attempt to respond. On an Ethernet, for example, using multiple RARP servers makes the probability of collision high.

How can the RARP service be arranged to keep it available and reliable without incurring the cost of multiple, simultaneous replies? There are at least two possibilities, and they both involve delaying responses. In the first solution, each machine that makes RARP requests is assigned a *primary server*. Under normal circumstances, only the machine's primary server responds to its RARP request. All nonprimary servers receive the request but merely record its arrival time. If the primary server is unavailable,

the original machine will timeout waiting for a response and then rebroadcast the request. Whenever a nonprimary server receives a second copy of a RARP request within a short time of the first, it responds.

The second solution uses a similar scheme but attempts to avoid having all nonprimary servers transmit responses simultaneously. Each nonprimary machine that receives a request computes a random delay and then sends a response. Under normal circumstances, the primary server responds immediately and successive responses are delayed, so there is low probability that several responses arrive at the same time. When the primary server is unavailable, the requesting machine experiences a small delay before receiving a reply. By choosing delays carefully, the designer can ensure that requesting machines do not rebroadcast before they receive an answer.

6.5 Summary

At system startup, a computer that does not have permanent storage must contact a server to find its IP address before it can communicate using TCP/IP. This chapter examined the RARP protocol that uses physical network addressing to obtain the machine's internet address. The RARP mechanism supplies the target machine's physical hardware address to uniquely identify the processor and broadcasts the RARP request. Servers on the network receive the message, look up the mapping in a table (presumably from secondary storage), and reply to the sender. Once a machine obtains its IP address, it stores the address in memory and does not use RARP again until it reboots.

FOR FURTHER STUDY

The details of RARP are given in Finlayson, *et. al.* [RFC 903]. Finlayson [RFC 906] describes workstation bootstrapping using the TFTP protocol. Bradley and Brown [RFC 1293] specifies a related protocol, *Inverse ARP*. Inverse ARP permits a computer to query the machine at the opposite end of a hardware connection to determine its IP address, and was intended for computers on a connection-oriented network such as Frame Relay or ATM. Volume 2 of this text describes an example implementation of RARP.

Chapter 23 considers alternatives to RARP known as BOOTP and DHCP. Unlike the low-level address determination scheme RARP supplies, BOOTP and DHCP build on higher level protocols like IP and UDP. Chapter 23 compares the two approaches, discussing the strengths and weaknesses of each.

EXERCISES

- 6.1** A RARP server can broadcast RARP replies to all machines or transmit each reply directly to the machine that makes the request. Characterize a network technology in which broadcasting replies to all machines is beneficial.
- 6.2** RARP is a narrowly focused protocol in the sense that replies only contain one piece of information (i.e., the requested IP address). When a computer boots, it usually needs to know its name in addition to its Internet address. Extend RARP to supply the additional information.
- 6.3** How much larger will Ethernet frames become when information is added to RARP as described in the previous exercise?
- 6.4** Adding a second RARP server to a network increases reliability. Does it ever make sense to add a third? How about a fourth? Why or Why not?
- 6.5** The diskless workstations from one vendor use RARP to obtain their IP addresses, but always assume the response comes from the workstation's file server. The diskless machine then tries to obtain a boot image from that server. If it does not receive a response, the workstation enters an infinite loop broadcasting boot requests. Explain how adding a backup RARP server to such a configuration can cause the network to become congested with broadcasts. Hint: think of power failures.
- 6.6** Monitor a local network while you reboot various computers. Which use RARP?
- 6.7** The backup RARP servers discussed in the text use the arrival of a second request in a short period of time to trigger a reply. Consider the RARP server scheme that has all servers answer the first request, but avoids congestion by having each server delay a random time before answering. Under what circumstances could such a design yield better results than the design described in the text?

Internet Protocol: Connectionless Datagram Delivery

7.1 Introduction

Previous chapters review pieces of network hardware and software that make internet communication possible, explaining the underlying network technologies and address resolution. This chapter explains the fundamental principle of connectionless delivery and discusses how it is provided by the *Internet Protocol (IP)*, which is one of the two major protocols used in internetworking (TCP being the other). We will study the format of IP datagrams and see how they form the basis for all internet communication. The next two chapters continue our examination of the Internet Protocol by discussing datagram routing and error handling.

7.2 A Virtual Network

Chapter 3 discusses an internet architecture in which routers connect multiple physical networks. Looking at the architecture may be misleading, because the focus should be on the interface that an internet provides to users, not on the interconnection technology.

A user thinks of an internet as a single virtual network that interconnects all hosts, and through which communication is possible; its underlying architecture is both hidden and irrelevant.

In a sense, an internet is an abstraction of physical networks because, at the lowest level, it provides the same functionality: accepting packets and delivering them. Higher levels of internet software add most of the rich functionality users perceive.

7.3 Internet Architecture And Philosophy

Conceptually, a TCP/IP internet provides three sets of services as shown in Figure 7.1; their arrangement in the figure suggests dependencies among them. At the lowest level, a connectionless delivery service provides a foundation on which everything rests. At the next level, a reliable transport service provides a higher level platform on which applications depend. We will soon explore each of these services, understand what they provide, and see the protocols associated with them.

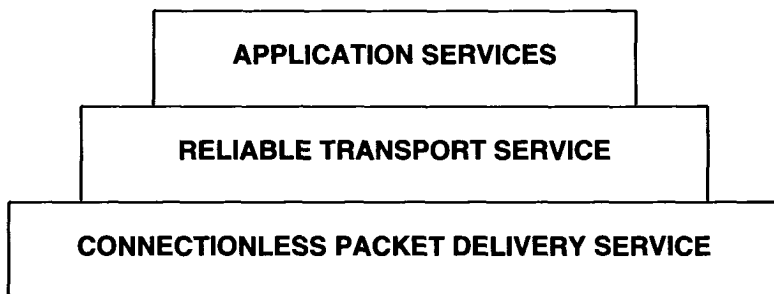


Figure 7.1 The three conceptual layers of internet services.

7.4 The Conceptual Service Organization

Although we can associate protocol software with each of the services in Figure 7.1, the reason for identifying them as conceptual parts of the internet is that they clearly point out the philosophical underpinnings of the design. The point is:

Internet software is designed around three conceptual networking services arranged in a hierarchy; much of its success has resulted because this architecture is surprisingly robust and adaptable.

One of the most significant advantages of this conceptual separation is that it becomes possible to replace one service without disturbing others. Thus, research and development can proceed concurrently on all three.

7.5 Connectionless Delivery System

The most fundamental internet service consists of a packet delivery system. Technically, the service is defined as an unreliable, best-effort, connectionless packet delivery system, analogous to the service provided by network hardware that operates on a best-effort delivery paradigm. The service is called *unreliable* because delivery is not guaranteed. The packet may be lost, duplicated, delayed, or delivered out of order, but the service will not detect such conditions, nor will it inform the sender or receiver. The service is called *connectionless* because each packet is treated independently from all others. A sequence of packets sent from one computer to another may travel over different paths, or some may be lost while others are delivered. Finally, the service is said to use *best-effort delivery* because the internet software makes an earnest attempt to deliver packets. That is, the internet does not discard packets capriciously; unreliability arises only when resources are exhausted or underlying networks fail.

7.6 Purpose Of The Internet Protocol

The protocol that defines the unreliable, connectionless delivery mechanism is called the *Internet Protocol* and is usually referred to by its initials, *IP*[†]. IP provides three important definitions. First, the IP protocol defines the basic unit of data transfer used throughout a TCP/IP internet. Thus, it specifies the exact format of all data as it passes across the internet. Second, IP software performs the *routing* function, choosing a path over which data will be sent. Third, in addition to the precise, formal specification of data formats and routing, IP includes a set of rules that embody the idea of unreliable packet delivery. The rules characterize how hosts and routers should process packets, how and when error messages should be generated, and the conditions under which packets can be discarded. IP is such a fundamental part of the design that a TCP/IP internet is sometimes called an *IP-based technology*.

We begin our consideration of IP in this chapter by looking at the packet format it specifies. We leave until later chapters the topics of routing and error handling.

7.7 The Internet Datagram

The analogy between a physical network and a TCP/IP internet is strong. On a physical network, the unit of transfer is a frame that contains a header and data, where the header gives information such as the (physical) source and destination addresses. The internet calls its basic transfer unit an *Internet datagram*, sometimes referred to as

[†]The abbreviation IP gives rise to the term “IP address.”

an *IP datagram* or merely a *datagram*. Like a typical physical network frame, a datagram is divided into header and data areas. Also like a frame, the datagram header contains the source and destination addresses and a type field that identifies the contents of the datagram. The difference, of course, is that the datagram header contains IP addresses whereas the frame header contains physical addresses. Figure 7.2 shows the general form of a datagram:



Figure 7.2 General form of an IP datagram, the TCP/IP analogy to a network frame. IP specifies the header format including the source and destination IP addresses. IP does not specify the format of the data area; it can be used to transport arbitrary data.

7.7.1 Datagram Format

Now that we have described the general layout of an IP datagram, we can look at the contents in more detail. Figure 7.3 shows the arrangement of fields in a datagram:

0	4	8	16	19	24	31
VERS	HLEN	SERVICE TYPE	TOTAL LENGTH			
IDENTIFICATION			FLAGS	FRAGMENT OFFSET		
TIME TO LIVE		PROTOCOL	HEADER CHECKSUM			
SOURCE IP ADDRESS						
DESTINATION IP ADDRESS						
IP OPTIONS (IF ANY)					PADDING	
DATA						
...						

Figure 7.3 Format of an Internet datagram, the basic unit of transfer in a TCP/IP internet.

Because datagram processing occurs in software, the contents and format are not constrained by any hardware. For example, the first 4-bit field in a datagram (*VERS*) contains the version of the IP protocol that was used to create the datagram. It is used to verify that the sender, receiver, and any routers in between them agree on the format

of the datagram. All IP software is required to check the version field before processing a datagram to ensure it matches the format the software expects. If standards change, machines will reject datagrams with protocol versions that differ from theirs, preventing them from misinterpreting datagram contents according to an outdated format. The current IP protocol version is 4. Consequently, the term *IPv4* is often used to denote the current protocol.

The header length field (*HLEN*), also 4 bits, gives the datagram header length measured in 32-bit words. As we will see, all fields in the header have fixed length except for the *IP OPTIONS* and corresponding *PADDING* fields. The most common header, which contains no options and no padding, measures 20 octets and has a header length field equal to 5.

The *TOTAL LENGTH* field gives the length of the IP datagram measured in octets, including octets in the header and data. The size of the data area can be computed by subtracting the length of the header (*HLEN*) from the *TOTAL LENGTH*. Because the *TOTAL LENGTH* field is 16 bits long, the maximum possible size of an IP datagram is 2^{16} or 65,535 octets. In most applications this is not a severe limitation. It may become more important in the future if higher speed networks can carry data packets larger than 65,535 octets.

7.7.2 Datagram Type Of Service And Differentiated Services

Informally called *Type Of Service (TOS)*, the 8-bit *SERVICE TYPE* field specifies how the datagram should be handled. The field was originally divided into five subfields as shown in Figure 7.4:



Figure 7.4 The original five subfields that comprise the 8-bit *SERVICE TYPE* field.

Three *PRECEDENCE* bits specify datagram precedence, with values ranging from 0 (normal precedence) through 7 (network control), allowing senders to indicate the importance of each datagram. Although some routers ignore type of service, it is an important concept because it provides a mechanism that can allow control information to have precedence over data. For example, many routers use a precedence value of 6 or 7 for routing traffic to make it possible for the routers to exchange routing information even when networks are congested.

Bits *D*, *T*, and *R* specify the type of transport desired for the datagram. When set, the *D* bit requests low delay, the *T* bit requests high throughput, and the *R* bit requests high reliability. Of course, it may not be possible for an internet to guarantee the type

of transport requested (i.e., it could be that no path to the destination has the requested property). Thus, we think of the transport request as a hint to the routing algorithms, not as a demand. If a router does know more than one possible route to a given destination, it can use the type of transport field to select one with characteristics closest to those desired. For example, suppose a router can select between a low capacity leased line or a high bandwidth (but high delay) satellite connection. Datagrams carrying keystrokes from a user to a remote computer could have the *D* bit set requesting that they be delivered as quickly as possible, while datagrams carrying a bulk file transfer could have the *T* bit set requesting that they travel across the high capacity satellite path.

In the late 1990s, the IETF redefined the meaning of the 8-bit *SERVICE TYPE* field to accommodate a set of *differentiated services (DS)*. Figure 7.5 illustrates the resulting definition.



Figure 7.5 The differentiated services (DS) interpretation of the *SERVICE TYPE* field in an IP datagram.

Under the differentiated services interpretation, the first six bits comprise a *codepoint*, which is sometimes abbreviated *DSCP*, and the last two bits are left unused. A codepoint value maps to an underlying service definition, typically through an array of pointers. Although it is possible to define 64 separate services, the designers suggest that a given router will only have a few services, and multiple codepoints will map to each service. Moreover, to maintain backward compatibility with the original definition, the standard distinguishes between the first three bits of the codepoint (the bits that were formerly used for precedence) and the last three bits. When the last three bits contain zero, the precedence bits define eight broad classes of service that adhere to the same guidelines as the original definition: datagrams with a higher number in their precedence field are given preferential treatment over datagrams with a lower number. That is, the eight ordered classes are defined by codepoint values of the form:

$$xxx000$$

where *x* denotes either a zero or a one.

The differentiated services design also accommodates another existing practice — the widespread use of precedence 6 or 7 for routing traffic. The standard includes a special case to handle these precedence values. A router is required to implement at least two priority schemes: one for normal traffic and one for high-priority traffic. When the last three bits of the *CODEPOINT* field are zero, the router must map a

codepoint with precedence 6 or 7 into the higher priority class and other codepoint values into the lower priority class. Thus, if a datagram arrives that was sent using the original TOS scheme, a router using the differentiated services scheme will honor precedence 6 and 7 as the datagram sender expects.

The 64 codepoint values are divided into three administrative groups as Figure 7.6 illustrates.

Pool	Codepoint	Assigned By
1	xxxxx0	Standards organization
2	xxxx11	Local or experimental
3	xxxx01	Local or experimental for now

Figure 7.6 The three administrative pools of codepoint values.

As the figure indicates, half of the values (i.e., the 32 values in pool 1) must be assigned interpretations by the IETF. Currently, all values in pools 2 and 3 are available for experimental or local use. However, if the standards bodies exhaust all values in pool 1, they may also choose to assign values in pool 3.

The division into pools may seem unusual because it relies on the low-order bits of the value to distinguish pools. Thus, rather than a contiguous set of values, pool 1 contains every other codepoint value (i.e., the even numbers between 2 and 64). The division was chosen to keep the eight codepoints corresponding to values xxx000 in the same pool.

Whether the original TOS interpretation or the revised differentiated services interpretation is used, it is important to realize that routing software must choose from among the underlying physical network technologies at hand and must adhere to local policies. Thus, specifying a level of service in a datagram does not guarantee that routers along the path will agree to honor the request. To summarize:

We regard the service type specification as a hint to the routing algorithm that helps it choose among various paths to a destination based on local policies and its knowledge of the hardware technologies available on those paths. An internet does not guarantee to provide any particular type of service.

7.7.3 Datagram Encapsulation

Before we can understand the next fields in a datagram, it is important to consider how datagrams relate to physical network frames. We start with a question: “How large can a datagram be?” Unlike physical network frames that must be recognized by hardware, datagrams are handled by software. They can be of any length the protocol designers choose. We have seen that the IPv4 datagram format allots 16 bits to the total length field, limiting the datagram to at most 65,535 octets.

More fundamental limits on datagram size arise in practice. We know that as datagrams move from one machine to another, they must always be transported by the underlying physical network. To make internet transportation efficient, we would like to guarantee that each datagram travels in a distinct physical frame. That is, we want our abstraction of a physical network packet to map directly onto a real packet if possible.

The idea of carrying one datagram in one network frame is called *encapsulation*. To the underlying network, a datagram is like any other message sent from one machine to another. The hardware does not recognize the datagram format, nor does it understand the IP destination address. Thus, as Figure 7.7 shows, when one machine sends an IP datagram to another, the entire datagram travels in the data portion of the network frame†.

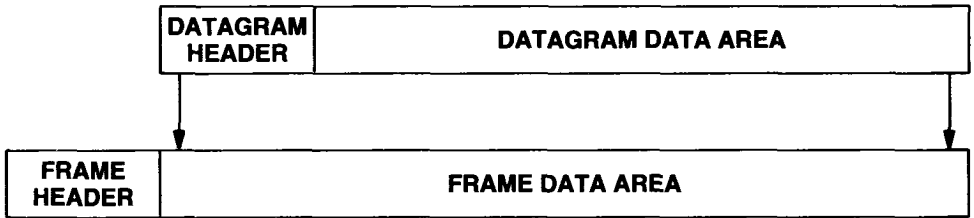


Figure 7.7 The encapsulation of an IP datagram in a frame. The physical network treats the entire datagram, including the header, as data.

7.7.4 Datagram Size, Network MTU, and Fragmentation

In the ideal case, the entire IP datagram fits into one physical frame, making transmission across the physical net efficient. To achieve such efficiency, the designers of IP might have selected a maximum datagram size such that a datagram would always fit into one frame. But which frame size should be chosen? After all, a datagram may travel across many types of physical networks as it moves across an internet to its final destination.

To understand the problem, we need a fact about network hardware: each packet-switching technology places a fixed upper bound on the amount of data that can be transferred in one physical frame. For example, Ethernet limits transfers to 1500 octets of data, while FDDI permits approximately 4470 octets of data per frame. We refer to these limits as the network's *maximum transfer unit* or *MTU*. MTU sizes can be quite small: some hardware technologies limit transfers to 128 octets or less. Limiting datagrams to fit the smallest possible MTU in the internet makes transfers inefficient when datagrams pass across a network that can carry larger size frames. However, allowing datagrams to be larger than the minimum network MTU in an internet means that a datagram may not always fit into a single network frame.

†A field in the frame header usually identifies the data being carried; Ethernet uses the type value 0800₁₆ to specify that the data area contains an encapsulated IP datagram.

The choice should be obvious: the point of the internet design is to hide underlying network technologies and make communication convenient for the user. Thus, instead of designing datagrams that adhere to the constraints of physical networks, TCP/IP software chooses a convenient initial datagram size and arranges a way to divide large datagrams into smaller pieces when the datagram needs to traverse a network that has a small MTU. The small pieces into which a datagram is divided are called *fragments*, and the process of dividing a datagram is known as *fragmentation*.

As Figure 7.8 illustrates, fragmentation usually occurs at a router somewhere along the path between the datagram source and its ultimate destination. The router receives a datagram from a network with a large MTU and must send it over a network for which the MTU is smaller than the datagram size.

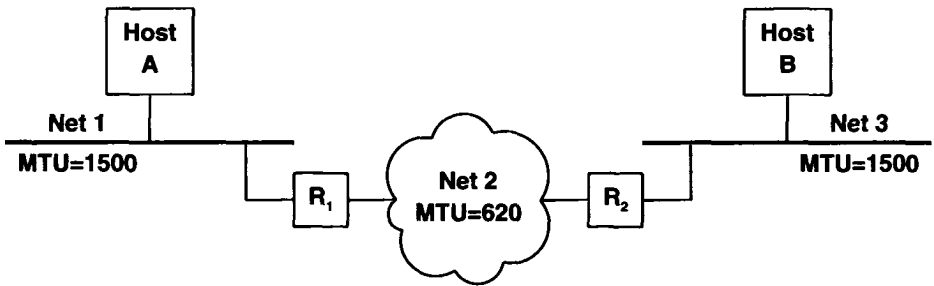


Figure 7.8 An illustration of where fragmentation occurs. Router R_1 fragments large datagrams sent from A to B; R_2 fragments large datagrams sent from B to A.

In the figure, both hosts attach directly to Ethernets which have an MTU of 1500 octets. Thus, both hosts can generate and send datagrams up to 1500 octets long. The path between them, however, includes a network with an MTU of 620. If host A sends host B a datagram larger than 620 octets, router R_1 will fragment the datagram. Similarly, if B sends a large datagram to A, router R_2 will fragment the datagram.

Fragment size is chosen so each fragment can be shipped across the underlying network in a single frame. In addition, because IP represents the offset of the data in multiples of eight octets, the fragment size must be chosen to be a multiple of eight. Of course, choosing the multiple of eight octets nearest to the network MTU does not usually divide the datagram into equal size pieces; the last piece is often shorter than the others. Fragments must be *reassembled* to produce a complete copy of the original datagram before it can be processed at the destination.

The IP protocol does not limit datagrams to a small size, nor does it guarantee that large datagrams will be delivered without fragmentation. The source can choose any datagram size it thinks appropriate; fragmentation and reassembly occur automatically, without the source taking special action. The IP specification states that routers must accept datagrams up to the maximum of the MTUs of networks to which they attach.

In addition, a router must always handle datagrams of up to 576 octets. (Hosts are also required to accept, and reassemble if necessary, datagrams of at least 576 octets.)

Fragmenting a datagram means dividing it into several pieces. It may surprise you to learn that each piece has the same format as the original datagram. Figure 7.9 illustrates the result of fragmentation.

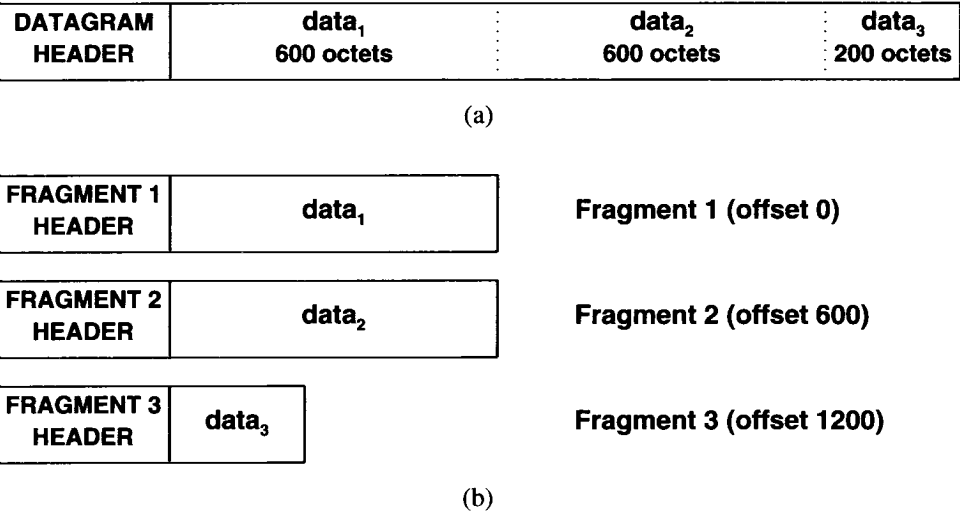


Figure 7.9 (a) An original datagram carrying 1400 octets of data and (b) the three fragments for network MTU of 620. Headers 1 and 2 have the *more fragments* bit set. Offsets shown are decimal octets; they must be divided by 8 to get the value stored in the fragment headers.

Each fragment contains a datagram header that duplicates most of the original datagram header (except for a bit in the *FLAGS* field that shows it is a fragment), followed by as much data as can be carried in the fragment while keeping the total length smaller than the MTU of the network over which it must travel.

7.7.5 Reassembly Of Fragments

Should a datagram be reassembled after passing across one network, or should the fragments be carried to the final host before reassembly? In a TCP/IP internet, once a datagram has been fragmented, the fragments travel as separate datagrams all the way to the ultimate destination where they must be reassembled. Preserving fragments all the way to the ultimate destination has two disadvantages. First, because datagrams are not reassembled immediately after passing across a network with small MTU, the small fragments must be carried from the point of fragmentation to the ultimate destination.

Reassembling datagrams at the ultimate destination can lead to inefficiency: even if some of the physical networks encountered after the point of fragmentation have large MTU capability, only small fragments traverse them. Second, if any fragments are lost, the datagram cannot be reassembled. The receiving machine starts a *reassembly timer* when it receives an initial fragment. If the timer expires before all fragments arrive, the receiving machine discards the surviving pieces without processing the datagram. Thus, the probability of datagram loss increases when fragmentation occurs because the loss of a single fragment results in loss of the entire datagram.

Despite the minor disadvantages, performing reassembly at the ultimate destination works well. It allows each fragment to be routed independently, and does not require intermediate routers to store or reassemble fragments.

7.7.6 Fragmentation Control

Three fields in the datagram header, *IDENTIFICATION*, *FLAGS*, and *FRAGMENT OFFSET*, control fragmentation and reassembly of datagrams. Field *IDENTIFICATION* contains a unique integer that identifies the datagram. Recall that when a router fragments a datagram, it copies most of the fields in the datagram header into each fragment. Thus, the *IDENTIFICATION* field must be copied. Its primary purpose is to allow the destination to know which arriving fragments belong to which datagrams. As a fragment arrives, the destination uses the *IDENTIFICATION* field along with the datagram source address to identify the datagram. Computers sending IP datagrams must generate a unique value for the *IDENTIFICATION* field for each datagram†. One technique used by IP software keeps a global counter in memory, increments it each time a new datagram is created, and assigns the result as the datagram's *IDENTIFICATION* field.

Recall that each fragment has exactly the same format as a complete datagram. For a fragment, field *FRAGMENT OFFSET* specifies the offset in the original datagram of the data being carried in the fragment, measured in units of 8 octets‡, starting at offset zero. To reassemble the datagram, the destination must obtain all fragments starting with the fragment that has offset 0 through the fragment with highest offset. Fragments do not necessarily arrive in order, and there is no communication between the router that fragmented the datagram and the destination trying to reassemble it.

The low-order two bits of the 3-bit *FLAGS* field control fragmentation. Usually, application software using TCP/IP does not care about fragmentation because both fragmentation and reassembly are automatic procedures that occur at a low level in the operating system, invisible to end users. However, to test internet software or debug operational problems, it may be important to test sizes of datagrams for which fragmentation occurs. The first control bit aids in such testing by specifying whether the datagram may be fragmented. It is called the *do not fragment* bit because setting it to 1 specifies that the datagram should not be fragmented. An application may choose to disallow fragmentation when only the entire datagram is useful. For example, consider a bootstrap sequence in which a small embedded system executes a program in ROM that sends a request over the internet to which another machine responds by sending

†In theory, retransmissions of a packet can carry the same *IDENTIFICATION* field as the original; in practice, higher-level protocols perform retransmission, resulting in a new datagram with its own *IDENTIFICATION*.

‡The same goes in the header: offsets are specified in multiples of 8 octets.

back a memory image. If the embedded system has been designed so it needs the entire image or none of it, the datagram should have the *do not fragment* bit set. Whenever a router needs to fragment a datagram that has the *do not fragment* bit set, the router discards the datagram and sends an error message back to the source.

The low order bit in the *FLAGS* field specifies whether the fragment contains data from the middle of the original datagram or from the end. It is called the *more fragments* bit. To see why such a bit is needed, consider the IP software at the ultimate destination attempting to reassemble a datagram. It will receive fragments (possibly out of order) and needs to know when it has received all fragments for a datagram. When a fragment arrives, the *TOTAL LENGTH* field in the header refers to the size of the fragment and not to the size of the original datagram, so the destination cannot use the *TOTAL LENGTH* field to tell whether it has collected all fragments. The *more fragments* bit solves the problem easily: once the destination receives a fragment with the *more fragments* bit turned off, it knows this fragment carries data from the tail of the original datagram. From the *FRAGMENT OFFSET* and *TOTAL LENGTH* fields, it can compute the length of the original datagram. By examining the *FRAGMENT OFFSET* and *TOTAL LENGTH* of all fragments that have arrived, a receiver can tell whether the fragments on hand contain all pieces needed to reassemble the original datagram.

7.7.7 Time to Live (TTL)

In principle, field *TIME TO LIVE* specifies how long, in seconds, the datagram is allowed to remain in the internet system. The idea is both simple and important: whenever a computer injects a datagram into the internet, it sets a maximum time that the datagram should survive. Routers and hosts that process datagrams must decrement the *TIME TO LIVE* field as time passes and remove the datagram from the internet when its time expires.

Estimating exact times is difficult because routers do not usually know the transit time for physical networks. A few rules simplify processing and make it easy to handle datagrams without synchronized clocks. First, each router along the path from source to destination is required to decrement the *TIME TO LIVE* field by 1 when it processes the datagram header. Furthermore, to handle cases of overloaded routers that introduce long delays, each router records the local time when the datagram arrives, and decrements the *TIME TO LIVE* by the number of seconds the datagram remained inside the router waiting for service†.

Whenever a *TIME TO LIVE* field reaches zero, the router discards the datagram and sends an error message back to the source. The idea of keeping a timer for datagrams is interesting because it guarantees that datagrams cannot travel around an internet forever, even if routing tables become corrupt and routers route datagrams in a circle.

Although once important, the notion of a router delaying a datagram for many seconds is now outdated — current routers and networks are designed to forward each datagram within a reasonable time. If the delay becomes excessive, the router simply discards the datagram. Thus, in practice, the *TIME TO LIVE* acts as a “hop limit” rather than an estimate of delay. Each router only decrements the value by 1.

†In practice, modern routers do not hold datagrams for multiple seconds.

7.7.8 Other Datagram Header Fields

Field *PROTOCOL* is analogous to the type field in a network frame; the value specifies which high-level protocol was used to create the message carried in the *DATA* area of the datagram. In essence, the value of *PROTOCOL* specifies the format of the *DATA* area. The mapping between a high level protocol and the integer value used in the *PROTOCOL* field must be administered by a central authority to guarantee agreement across the entire Internet.

Field *HEADER CHECKSUM* ensures integrity of header values. The IP checksum is formed by treating the header as a sequence of 16-bit integers (in network byte order), adding them together using one's complement arithmetic, and then taking the one's complement of the result. For purposes of computing the checksum, field *HEADER CHECKSUM* is assumed to contain zero.

It is important to note that the checksum only applies to values in the IP header and not to the data. Separating the checksum for headers and data has advantages and disadvantages. Because the header usually occupies fewer octets than the data, having a separate checksum reduces processing time at routers which only need to compute header checksums. The separation also allows higher level protocols to choose their own checksum scheme for the data. The chief disadvantage is that higher level protocols are forced to add their own checksum or risk having corrupted data go undetected.

Fields *SOURCE IP ADDRESS* and *DESTINATION IP ADDRESS* contain the 32-bit IP addresses of the datagram's sender and intended recipient. Although the datagram may be routed through many intermediate routers, the source and destination fields never change; they specify the IP addresses of the original source and ultimate destination†.

The field labeled *DATA* in Figure 7.3 shows the beginning of the data area of the datagram. Its length depends, of course, on what is being sent in the datagram. The *IP OPTIONS* field, discussed below, is variable length. The field labeled *PADDING*, depends on the options selected. It represents bits containing zero that may be needed to ensure the datagram header extends to an exact multiple of 32 bits (recall that the header length field is specified in units of 32-bit words).

7.8 Internet Datagram Options

The *IP OPTIONS* field following the destination address is not required in every datagram; options are included primarily for network testing or debugging. Options processing is an integral part of the IP protocol, however, so all standard implementations must include it.

The length of the *IP OPTIONS* field varies depending on which options are selected. Some options are one octet long; they consist of a single octet *option code*. Other options are variable length. When options are present in a datagram, they appear contiguously, with no special separators between them. Each option consists of a single octet option code, which may be followed by a single octet length and a set of data octets for that option. The option code octet is divided into three fields as Figure 7.10 shows.

†An exception is made when the datagram includes the source route options listed below.

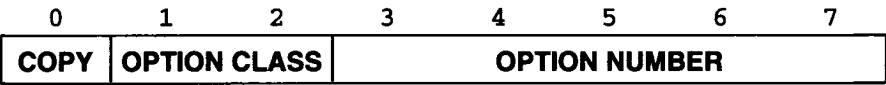


Figure 7.10 The division of the option code octet into three fields of length 1, 2, and 5 bits.

The fields of the *OPTION CODE* consist of a 1-bit *COPY* flag, a 2-bit *OPTION CLASS*, and the 5-bit *OPTION NUMBER*. The *COPY* flag controls how routers treat options during fragmentation. When the *COPY* bit is set to 1, it specifies that the option should be copied into all fragments. When set to 0, the *COPY* bit means that the option should only be copied into the first fragment and not into all fragments.

The *OPTION CLASS* and *OPTION NUMBER* bits specify the general class of the option and a specific option in that class. The table in Figure 7.11 shows how option classes are assigned.

Option Class	Meaning
0	Datagram or network control
1	Reserved for future use
2	Debugging and measurement
3	Reserved for future use

Figure 7.11 Classes of IP options as encoded in the *OPTION CLASS* bits of an option code octet.

The table in Figure 7.12 lists examples of options that can accompany an IP datagram and gives their *OPTION CLASS* and *OPTION NUMBER* values. As the list shows, most options are used for control purposes.

Option Class	Option Number	Length	Description
0	0	-	End of option list. Used if options do not end at end of header (see header padding field for explanation).
0	1	-	No operation. Used to align octets in a list of options.
0	2	11	Security and handling restrictions (for military applications).
0	3	var	Loose source route. Used to request routing that includes the specified routers.
0	7	var	Record route. Used to trace a route.
0	8	4	Stream identifier. Used to carry a SATNET stream identifier (obsolete).
0	9	var	Strict source route. Used to specify a exact path through the internet.
0	11	4	MTU Probe. Used for path MTU discovery.
0	12	4	MTU Reply. Used for path MTU discovery.
0	20	4	Router Alert. Router should examine this datagram even if not an addressee.
2	4	var	Internet timestamp. Used to record timestamps along the route.
2	18	var	Traceroute. Used by traceroute program to find routers along a path.

Figure 7.12 Examples of IP options with their numeric class and number codes. The value *var* in the length column stands for *variable*.

7.8.1 Record Route Option

The routing and timestamp options are the most interesting because they provide a way to monitor or control how internet routers route datagrams. The *record route* option allows the source to create an empty list of IP addresses and arrange for each router that handles the datagram to add its IP address to the list. Figure 7.13 shows the format of the record route option.

As described above, the *CODE* field contains the option class and option number (0 and 7 for record route). The *LENGTH* field specifies the total length of the option as it appears in the IP datagram, including the first three octets. The fields starting with the one labeled *FIRST IP ADDRESS* comprise the area reserved for recording internet addresses. The *POINTER* field specifies the offset within the option of the next available slot.

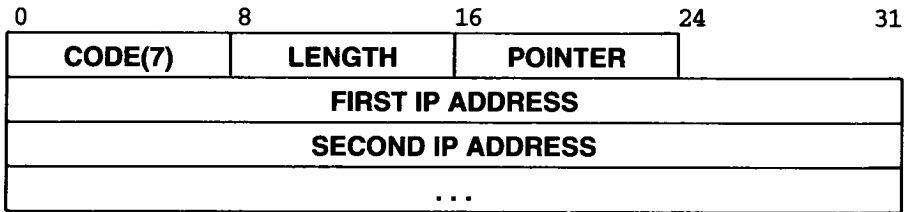


Figure 7.13 The format of the record route option in an IP datagram. The option begins with three octets immediately followed by a list of addresses. Although the diagram shows addresses in 32 bit units, they are not aligned on any octet boundary in a datagram.

Whenever a machine handles a datagram that has the record route option set, the machine adds its address to the record route list (enough space must be allocated in the option by the original source to hold all entries that will be needed). To add itself to the list, a machine first compares the pointer and length fields. If the pointer is greater than the length, the list is full, so the machine forwards the datagram without inserting its entry. If the list is not full, the machine inserts its 4-octet IP address at the position specified by the *POINTER*, and increments the *POINTER* by four.

When the datagram arrives, the destination machine can extract and process the list of IP addresses. Usually, a computer that receives a datagram ignores the recorded route. Using the record route option requires two machines that agree to cooperate; a computer will not automatically receive recorded routes in incoming datagrams after it turns on the record route option in outgoing datagrams. The source must agree to enable the record route option and the destination must agree to process the resultant list.

7.8.2 Source Route Options

Another idea that network builders find interesting is the *source route* option. The idea behind source routing is that it provides a way for the sender to dictate a path through the internet. For example, to test the throughput over a particular physical network, *N*, system administrators can use source routing to force IP datagrams to traverse network *N* even if routers would normally choose a path that did not include it. The ability to make such tests is especially important in a production environment, because it gives the network manager freedom to route users' datagrams over networks that are known to operate correctly while simultaneously testing other networks. Of course, source routing is only useful to people who understand the network topology; the average user has no need to know or use it.

IP supports two forms of source routing. One form, called *strict source routing*, specifies a routing path by including a sequence of IP addresses in the option as Figure 7.14 shows.

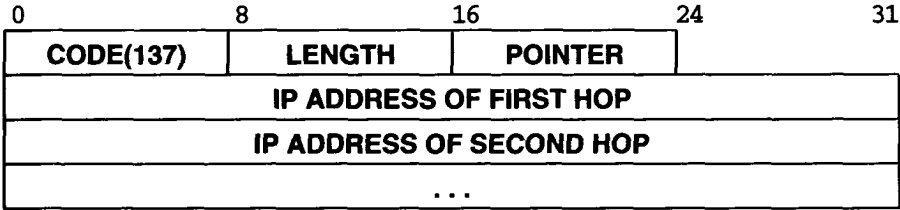


Figure 7.14 The strict source route option specifies an exact route by giving a list of IP addresses the datagram must follow.

Strict source routing means that the addresses specify the exact path the datagram must follow to reach its destination. The path between two successive addresses in the list must consist of a single physical network; an error results if a router cannot follow a strict source route. The other form, called *loose source routing*, also includes a sequence of IP addresses. It specifies that the datagram must follow the sequence of IP addresses, but allows multiple network hops between successive addresses on the list.

Both source route options require routers along the path to overwrite items in the address list with their local network addresses. Thus, when the datagram arrives at its destination, it contains a list of all addresses visited, exactly like the list produced by the record route option.

The format of a source route option resembles that of the record route option shown above. Each router examines the *POINTER* and *LENGTH* fields to see if the list has been exhausted. If it has, the pointer is greater than the length, and the router routes the datagram to its destination as usual. If the list is not exhausted, the router follows the pointer, picks up the IP address, replaces it with the router's address†, and routes the datagram using the address obtained from the list.

7.8.3 Timestamp Option

The *timestamp option* works like the record route option in that the timestamp option contains an initially empty list, and each router along the path from source to destination fills in one item in the list. Each entry in the list contains two 32-bit items: the IP address of the router that supplied the entry and a 32-bit integer timestamp. Figure 7.15 shows the format of the timestamp option.

†A router has one address for each interface; it records the address that corresponds to the network over which it routes the datagram.

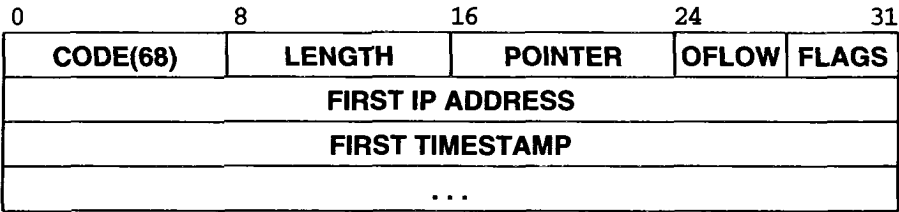


Figure 7.15 The format of the timestamp option. Bits in the *FLAGS* field control the exact format and rules routers use to process this option.

In the figure, the *LENGTH* and *POINTER* fields are used to specify the length of the space reserved for the option and the location of the next unused slot (exactly as in the record route option). The 4-bit *OFLOW* field contains an integer count of routers that could not supply a timestamp because the option was too small.

The value in the 4-bit *FLAGS* field controls the exact format of the option and tells how routers should supply timestamps. The values are:

Flags value	Meaning
0	Record timestamps only; omit IP addresses.
1	Precede each timestamp by an IP address (this is the format shown in Figure 7.15).
3	IP addresses are specified by sender; a router only records a timestamp if the next IP address in the list matches the router's IP address.

Figure 7.16 The interpretation of values in the *FLAGS* field of a timestamp option.

Timestamps give the time and date at which a router handles the datagram, expressed as milliseconds since midnight, Universal Time†. If the standard representation for time is unavailable, the router can use any representation of local time provided it turns on the high-order bit in the timestamp field. Of course, timestamps issued by independent computers are not always consistent even if represented in universal time; each machine reports time according to its local clock, and clocks may differ. Thus, timestamp entries should always be treated as estimates, independent of the representation.

It may seem odd that the timestamp option includes a mechanism to have routers record their IP addresses along with timestamps because the record route option already provides that capability. However, recording IP addresses with timestamps eliminates

† Universal Time was formerly called Greenwich Mean Time; it is the time of day at the prime meridian.

ambiguity. Having an address recorded along with each timestamp is also useful because it allows the receiver to know exactly which path the datagram followed.

7.8.4 Processing Options During Fragmentation

The idea behind the *COPY* bit in the option *CODE* field should now be clear. When fragmenting a datagram, a router replicates some IP options in all fragments while it places others in only one fragment. For example, consider the option used to record the datagram route. We said that each fragment will be handled as an independent datagram, so there is no guarantee that all fragments follow the same path to the destination. If all fragments contained the record route option, the destination might receive a different list of routes from each fragment. It could not produce a single, meaningful list of routes for the reassembled datagram. Therefore, the IP standard specifies that the record route option should only be copied into one of the fragments.

Not all IP options can be restricted to one fragment. Consider the source route option, for example, that specifies how a datagram should travel through the internet. Source routing information must be replicated in all fragment headers, or fragments will not follow the specified route. Thus, the code field for source route specifies that the option must be copied into all fragments.

7.9 Summary

The fundamental service provided by TCP/IP internet software is a connectionless, unreliable, best-effort packet delivery system. The Internet Protocol (IP) formally specifies the format of internet packets, called *datagrams*, and informally embodies the ideas of connectionless delivery. This chapter concentrated on datagram formats; later chapters will discuss IP routing and error handling.

Analogous to a physical frame, the IP datagram is divided into header and data areas. Among other information, the datagram header contains the source and destination IP addresses, fragmentation control, precedence, and a checksum used to catch transmission errors. Besides fixed-length fields, each datagram header can contain an options field. The options field is variable length, depending on the number and type of options used as well as the size of the data area allocated for each option. Intended to help monitor and control an internet, options allow one to specify or record routing information, or to gather timestamps as the datagram traverses an internet.

FOR FURTHER STUDY

Postel [1980] discusses possible ways to approach internet protocols, addressing, and routing. In later publications, Postel [RFC 791] gives the standard for the Internet Protocol. Braden [RFC 1122] further refines the standard. Hornig [RFC 894] specifies

the standard for the transmission of IP datagrams across an Ethernet. Clark [RFC 815] describes efficient reassembly of fragments; Kent and Mogul [1987] discusses the disadvantages of fragmentation.

Nichols et. al. [RFC 2474] specifies the differentiated service interpretation of the service type bits in datagram headers, and Blake et. al. [RFC 2475] discusses an architecture for differentiated services. In addition to the packet format, many constants needed in the network protocols are also standardized; the values can be found in the Official Internet Protocols RFC, which is issued periodically.

An alternative internet protocol suite known as *XNS*, is given in Xerox [1981]. Boggs et. al. [1980] describes the PARC Universal Packet (PUP) protocol, an abstraction from *XNS* closely related to the IP datagram.

EXERCISES

- 7.1 What is the single greatest advantage of having the IP checksum cover only the datagram header and not the data? What is the disadvantage?
- 7.2 Is it ever necessary to use an IP checksum when sending packets over an Ethernet? Why or why not?
- 7.3 What is the MTU size for a Frame Relay network? Hyperchannel? an ATM network?
- 7.4 Do you expect a high-speed local area network to have larger or smaller MTU size than a wide area network?
- 7.5 Argue that fragments should have small, nonstandard headers.
- 7.6 Find out when the IP protocol version last changed. Is having a protocol version number useful?
- 7.7 Extend the previous exercise by arguing that if the IP version changes, it makes more sense to assign a new frame type than to encode the version number in the datagram.
- 7.8 Can you imagine why a one's complement checksum was chosen for IP instead of a cyclic redundancy check?
- 7.9 What are the advantages of doing reassembly at the ultimate destination instead of doing it after the datagram travels across one network?
- 7.10 What is the minimum network MTU required to send an IP datagram that contains at least one octet of data?
- 7.11 Suppose you are hired to implement IP datagram processing in hardware. Is there any rearrangement of fields in the header that would have made your hardware more efficient? Easier to build?
- 7.12 If you have access to an implementation of IP, revise it and test your locally available implementations of IP to see if they reject IP datagrams with an out-of-date version number.
- 7.13 When a minimum-size IP datagram travels across an Ethernet, how large is the frame?
- 7.14 The differentiated services interpretation of the *SERVICE TYPE* field allows up to 64 separate service levels. Argue that fewer levels are needed (i.e., make a list of all possible services that a user might access).
- 7.15 The differentiated service definition was chosen to make it backward compatible with the original type-of-service priority bits. Will the backward compatibility force implementations to be less efficient than an alternative scheme? Explain.

8

Internet Protocol: Routing IP Datagrams

8.1 Introduction

We have seen that all internet services use an underlying, connectionless packet delivery system, and that the basic unit of transfer in a TCP/IP internet is the IP datagram. This chapter adds to the description of connectionless service by describing how routers forward IP datagrams and deliver them to their final destinations. We think of the datagram format from Chapter 7 as characterizing the static aspects of the Internet Protocol. The description of routing in this chapter characterizes the operational aspects. The next chapter completes our basic presentation of IP by describing how errors are handled. Chapter 10 then describes extensions for classless and subnet addressing, and later chapters show how other protocols use IP to provide higher-level services.

8.2 Routing In An Internet

In a packet switching system, *routing* refers to the process of choosing a path over which to send packets, and *router* refers to a computer making the choice. Routing occurs at several levels. For example, within a wide area network that has multiple physical connections between packet switches, the network itself is responsible for routing packets from the time they enter until they leave. Such internal routing is completely self-contained inside the wide area network. Machines on the outside cannot participate in decisions; they merely view the network as an entity that delivers packets.

Remember that the goal of IP is to provide a virtual network that encompasses multiple physical networks and offers a connectionless datagram delivery service. Thus, we will focus on *IP forwarding*, which is also called *internet routing* or *IP routing*[†]. The information used to make routing decisions is known as *IP routing information*. Like routing within a single physical network, IP routing chooses a path over which a datagram should be sent. Unlike routing within a single network, the IP routing algorithm must choose how to send a datagram across multiple physical networks.

Routing in an internet can be difficult, especially among computers that have multiple physical network connections. Ideally, the routing software would examine network load, datagram length, or the type of service specified in the datagram header when selecting the best path. Most internet routing software is much less sophisticated, however, and selects routes based on fixed assumptions about shortest paths.

To understand IP routing completely, we must review the architecture of a TCP/IP internet. First, recall that an internet is composed of multiple physical networks interconnected by computers called *routers*. Each router has direct connections to two or more networks. By contrast, a host computer usually connects directly to one physical network. We know that it is possible, however, to have a multi-homed host connected directly to multiple networks.

Both hosts and routers participate in routing an IP datagram to its destination. When an application program on a host attempts to communicate, the TCP/IP protocols eventually generate one or more IP datagrams. The host must make an initial routing decision when it chooses where to send the datagrams. As Figure 8.1 shows, hosts must make routing decisions even if they have only one network connection.

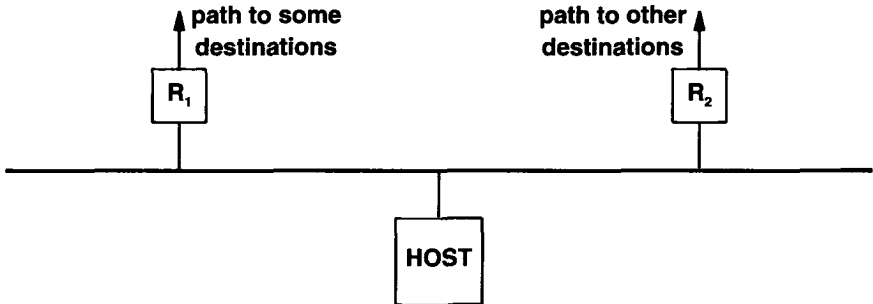


Figure 8.1 An example of a singly-homed host that must route datagrams. The host must choose to send a datagram either to router R_1 or to router R_2 , because each router provides the best path to some destinations.

The primary purpose of routers is to make IP routing decisions. What about multi-homed hosts? Any computer with multiple network connections can act as a router, and as we will see, multi-homed hosts running TCP/IP have all the software

[†]Chapter 18 describes a related topic known as *layer 3 switching* or *IP switching*.

needed for routing. Furthermore, sites that cannot afford separate routers sometimes use general-purpose timesharing machines as both hosts and routers. However, the TCP/IP standards draw a sharp distinction between the functions of a host and those of a router, and sites that try to mix host and router functions on a single machine sometimes find that their multi-homed hosts engage in unexpected interactions. For now, we will distinguish hosts from routers, and assume that hosts do not perform the router's function of transferring packets from one network to another.

8.3 Direct And Indirect Delivery

Loosely speaking, we can divide routing into two forms: *direct delivery* and *indirect delivery*. Direct delivery, the transmission of a datagram from one machine across a single physical network directly to another, is the basis on which all internet communication rests. Two machines can engage in direct delivery only if they both attach directly to the same underlying physical transmission system (e.g., a single Ethernet). *Indirect delivery* occurs when the destination is not on a directly attached network, forcing the sender to pass the datagram to a router for delivery.

8.3.1 Datagram Delivery Over A Single Network

We know that one machine on a given physical network can send a physical frame directly to another machine on the same network. To transfer an IP datagram, the sender encapsulates the datagram in a physical frame, maps the destination IP address into a physical address, and uses the network hardware to deliver it. Chapter 5 presented two possible mechanisms for address resolution, including using the ARP protocol for dynamic address binding on Ethernet-like networks. Chapter 7 discussed datagram encapsulation. Thus, we have reviewed all the pieces needed to understand direct delivery. To summarize:

Transmission of an IP datagram between two machines on a single physical network does not involve routers. The sender encapsulates the datagram in a physical frame, binds the destination IP address to a physical hardware address, and sends the resulting frame directly to the destination.

How does the sender know whether the destination lies on a directly connected network? The test is straightforward. We know that IP addresses are divided into a network-specific prefix and a host-specific suffix. To see if a destination lies on one of the directly connected networks, the sender extracts the network portion of the destination IP address and compares it to the network portion of its own IP address(es). A match means the datagram can be sent directly. Here we see one of the advantages of the Internet address scheme, namely:

Because the internet addresses of all machines on a single network include a common network prefix and extracting that prefix requires only a few machine instructions, testing whether a machine can be reached directly is extremely efficient.

From an internet perspective, it is easiest to think of direct delivery as the final step in any datagram transmission, even if the datagram traverses many networks and intermediate routers. The final router along the path between the datagram source and its destination will connect directly to the same physical network as the destination. Thus, the final router will deliver the datagram using direct delivery. We can think of direct delivery between the source and destination as a special case of general purpose routing – in a direct route the datagram does not happen to pass through any intervening routers.

8.3.2 Indirect Delivery

Indirect delivery is more difficult than direct delivery because the sender must identify a router to which the datagram can be sent. The router must then forward the datagram on toward its destination network.

To visualize how indirect routing works, imagine a large internet with many networks interconnected by routers but with only two hosts at the far ends. When one host wants to send to the other, it encapsulates the datagram and sends it to the nearest router. We know that the host can reach a router because all physical networks are interconnected, so there must be a router attached to each network. Thus, the originating host can reach a router using a single physical network. Once the frame reaches the router, software extracts the encapsulated datagram, and the IP software selects the next router along the path towards the destination. The datagram is again placed in a frame and sent over the next physical network to a second router, and so on, until it can be delivered directly. These ideas can be summarized:

Routers in a TCP/IP internet form a cooperative, interconnected structure. Datagrams pass from router to router until they reach a router that can deliver the datagram directly.

How can a router know where to send each datagram? How can a host know which router to use for a given destination? The two questions are related because they both involve IP routing. We will answer them in two stages, considering the basic table-driven routing algorithm in this chapter and postponing a discussion of how routers learn new routes until later.

8.4 Table-Driven IP Routing

The usual IP routing algorithm employs an *Internet routing table* (sometimes called an *IP routing table*) on each machine that stores information about possible destinations and how to reach them. Because both hosts and routers route datagrams, both have IP routing tables. Whenever the IP routing software in a host or router needs to transmit a datagram, it consults the routing table to decide where to send the datagram.

What information should be kept in routing tables? If every routing table contained information about every possible destination address, it would be impossible to keep the tables current. Furthermore, because the number of possible destinations is large, machines would have insufficient space to store the information.

Conceptually, we would like to use the principle of information hiding and allow machines to make routing decisions with minimal information. For example, we would like to isolate information about specific hosts to the local environment in which they exist and arrange for machines that are far away to route packets to them without knowing such details. Fortunately, the IP address scheme helps achieve this goal. Recall that IP addresses are assigned to make all machines connected to a given physical network share a common prefix (the network portion of the address). We have already seen that such an assignment makes the test for direct delivery efficient. It also means that routing tables only need to contain network prefixes and not full IP addresses.

8.5 Next-Hop Routing

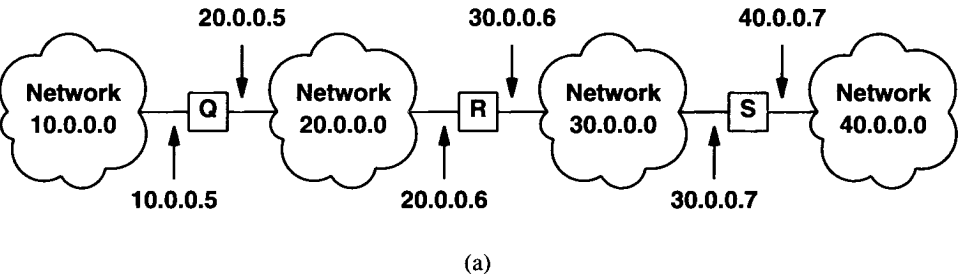
Using the network portion of a destination address instead of the complete host address makes routing efficient and keeps routing tables small. More important, it helps hide information, keeping the details of specific hosts confined to the local environment in which those hosts operate. Typically, a routing table contains pairs (N, R) , where N is the IP address of a destination *network*, and R is the IP address of the “next” router along the path to network N . Router R is called the *next hop*, and the idea of using a routing table to store a next hop for each destination is called *next-hop routing*. Thus, the routing table in a router R only specifies one step along the path from R to a destination network – the router does not know the complete path to a destination.

It is important to understand that each entry in a routing table points to a router that can be reached across a single network. That is, all routers listed in machine M 's routing table must lie on networks to which M connects directly. When a datagram is ready to leave M , IP software locates the destination IP address and extracts the network portion. M then uses the network portion to make a routing decision, selecting a router that can be reached directly.

In practice, we apply the principle of information hiding to hosts as well. We insist that although hosts have IP routing tables, they must keep minimal information in their tables. The idea is to force hosts to rely on routers for most routing.

Figure 8.2 shows a concrete example that helps explain routing tables. The example internet consists of four networks connected by three routers. In the figure, the rout-

ing table gives the routes that router *R* uses. Because *R* connects directly to networks 20.0.0.0 and 30.0.0.0, it can use direct delivery to send to a host on either of those networks (possibly using ARP to find physical addresses). Given a datagram destined for a host on network 40.0.0.0, *R* routes it to the address of router *S*, 30.0.0.7. *S* will then deliver the datagram directly. *R* can reach address 30.0.0.7 because both *R* and *S* attach directly to network 30.0.0.0.



TO REACH HOSTS ON NETWORK	ROUTE TO THIS ADDRESS
20.0.0.0	DELIVER DIRECTLY
30.0.0.0	DELIVER DIRECTLY
10.0.0.0	20.0.0.5
40.0.0.0	30.0.0.7

(b)

Figure 8.2 (a) An example internet with 4 networks and 3 routers, and (b) the routing table in *R*.

As Figure 8.2 demonstrates, the size of the routing table depends on the number of networks in the internet; it only grows when new networks are added. However, the table size and contents are independent of the number of individual hosts connected to the networks. We can summarize the underlying principle:

To hide information, keep routing tables small, and make routing decisions efficient, IP routing software only keeps information about destination network addresses, not about individual host addresses.

Choosing routes based on the destination network ID alone has several consequences. First, in most implementations, it means that all traffic destined for a given network takes the same path. As a result, even when multiple paths exist, they may not be used concurrently. Also, all types of traffic follow the same path without regard to the delay or throughput of physical networks. Second, because only the final router along the path attempts to communicate with the destination host, only it can determine if the host exists or is operational. Thus, we need to arrange a way for that router to send reports of delivery problems back to the original source. Third, because each router forwards traffic independently, datagrams traveling from host *A* to host *B* may follow an entirely different path than datagrams traveling from host *B* back to host *A*. We need to ensure that routers cooperate to guarantee that two-way communication is always possible.

8.6 Default Routes

Another technique used to hide information and keep routing table sizes small consolidates multiple entries into a default case. The idea is to have the IP routing software first look in the routing table for the destination network. If no route appears in the table, the routing routines send the datagram to a *default router*.

Default routing is especially useful when a site has a small set of local addresses and only one connection to the rest of the internet. For example, default routes work well in host computers that attach to a single physical network and reach only one router leading to the remainder of the internet. The routing decision consists of two tests: one for the local net and a default that points to the only router. Even if the site contains a few local networks, the routing is simple because it consists of a few tests for the local networks plus a default for all other destinations.

8.7 Host-Specific Routes

Although we said that all routing is based on networks and not on individual hosts, most IP routing software allows per-host routes to be specified as a special case. Having per-host routes gives the local network administrator more control over network use, permits testing, and can also be used to control access for security purposes. When debugging network connections or routing tables, the ability to specify a special route to one individual machine turns out to be especially useful.

8.8 The IP Routing Algorithm

Taking into account everything we have said, the IP algorithm used to forward datagrams becomes†:

†Chapter 10 discusses a slightly modified algorithm used with classless IP addresses.

Algorithm:**RouteDatagram (Datagram , RoutingTable)**

**Extract destination IP address, D, from the datagram
and compute the network prefix, N;
if N matches any directly connected network address
 deliver datagram to destination D over that network
 (This involves resolving D to a physical address,
 encapsulating the datagram, and sending the frame.)
else if the table contains a host-specific route for D
 send datagram to next-hop specified in table
else if the table contains a route for network N
 send datagram to next-hop specified in table
else if the table contains a default route
 send datagram to the default router specified in table
else declare a routing error;**

Figure 8.3 The algorithm IP uses to forward a datagram. Given an IP datagram and a routing table, this algorithm selects the next hop to which the datagram should be sent. All routes must specify a next hop that lies on a directly connected network.

8.9 Routing With IP Addresses

It is important to understand that except for decrementing the time to live and recomputing the checksum, IP routing does not alter the original datagram. In particular, the datagram source and destination addresses remain unaltered; they always specify the IP address of the original source and the IP address of the ultimate destination[†]. When IP executes the routing algorithm, it selects a new IP address, the IP address of the machine to which the datagram should be sent next. The new address is most likely the address of a router. However, if the datagram can be delivered directly, the new address is the same as the address of the ultimate destination.

We said that the IP address selected by the IP routing algorithm is known as the *next hop* address because it tells where the datagram must be sent next. Where does IP store the next hop address? Not in the datagram; no place is reserved for it. In fact, IP does not “store” the next hop address at all. After executing the routing algorithm, IP passes the datagram and the next hop address to the network interface software responsible for the physical network over which the datagram must be sent. The network in-

[†]The only exception occurs when the datagram contains a source route option.

terface software binds the next hop address to a physical address, forms a frame using that physical address, places the datagram in the data portion of the frame, and sends the result. After using the next hop address to find a physical address, the network interface software discards the next hop address.

It may seem odd that routing tables store the IP address of a next hop for each destination network when those addresses must be translated into corresponding physical addresses before the datagram can be sent. If we imagine a host sending a sequence of datagrams to the same destination address, the use of IP addresses will appear incredibly inefficient. IP dutifully extracts the destination address in each datagram and uses the routing table to produce a next hop address. It then passes the datagram and next hop address to the network interface, which recomputes the binding to a physical address. If the routing table used physical addresses, the binding between the next hop's IP address and physical address could be performed once, saving unneeded computation.

Why does IP software avoid using physical addresses when storing and computing routes? As Figure 8.4 illustrates, there are two important reasons.

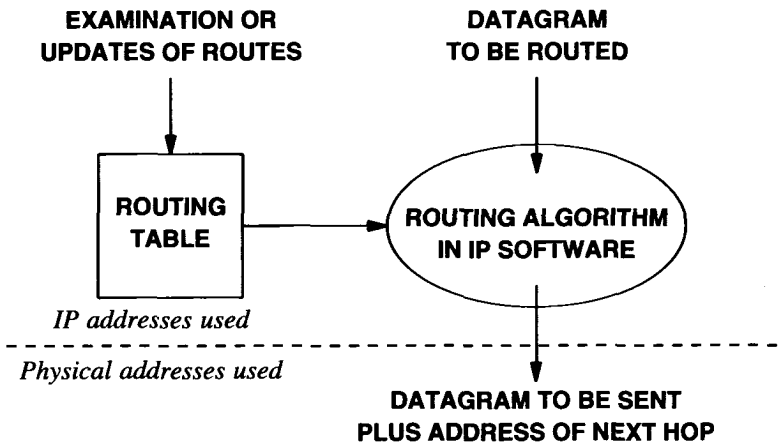


Figure 8.4 IP software and the routing table it uses reside above the address boundary. Using only IP addresses makes routes easy to examine or change and hides the details of physical addresses.

First, the routing table provides an especially clean interface between IP software that routes datagrams and high-level software that manipulates routes. To debug routing problems, network managers often need to examine the routing tables. Using only IP addresses in the routing table makes it easy for managers to understand and to determine whether software has updated the routes correctly. Second, the whole point of the Internet Protocol is to build an abstraction that hides the details of underlying networks.

Figure 8.4 shows the *address boundary*, the important conceptual division between low-level software that understands physical addresses and internet software that only uses high-level addresses. Above this boundary, all software can be written to communicate using internet addresses; knowledge of physical addresses is relegated to a few small, low-level routines. We will see that observing the boundary also helps keep the implementation of remaining TCP/IP protocols easy to understand, test, and modify.

8.10 Handling Incoming Datagrams

So far, we have discussed IP routing by describing how forwarding decisions are made about outgoing packets. It should be clear, however, that IP software must process incoming datagrams as well.

When an IP datagram arrives at a host, the network interface software delivers it to the IP module for processing. If the datagram's destination address matches the host's IP address, IP software on the host accepts the datagram and passes it to the appropriate higher-level protocol software for further processing. If the destination IP address does not match, a host is required to discard the datagram (i.e., hosts are forbidden from attempting to forward datagrams that are accidentally routed to the wrong machine).

Unlike hosts, routers perform forwarding. When an IP datagram arrives at a router, it is delivered to the IP software. Again, two cases arise: the datagram could have reached its final destination, or it may need to travel further. As with hosts, if the datagram destination IP address matches the router's own IP address, the IP software passes the datagram to higher-level protocol software for processing†. If the datagram has not reached its final destination, IP routes the datagram using the standard algorithm and the information in the local routing table.

Determining whether an IP datagram has reached its final destination is not quite as trivial as it seems. Remember that even a host may have multiple physical connections, each with its own IP address. When an IP datagram arrives, the machine must compare the destination internet address to the IP address for each of its network connections. If any match, it keeps the datagram and processes it. A machine must also accept datagrams that were broadcast on the physical network if their destination IP address is the limited IP broadcast address or the directed IP broadcast address for that network. As we will see in Chapters 10 and 17, classless, subnet, and multicast addresses make address recognition even more complex. In any case, if the address does not match any of the local machine's addresses, IP decrements the time-to-live field in the datagram header, discarding the datagram if the count reaches zero, or computing a new checksum and routing the datagram if the count remains positive.

Should every machine forward the IP datagrams it receives? Obviously, a router must forward incoming datagrams because that is its main function. We have also said that some multi-homed hosts act as routers even though they are really general purpose computing systems. While using a host as a router is not usually a good idea, if one chooses to use that arrangement, the host must be configured to route datagrams just as a router does. But what about other hosts, those that are not intended to be routers?

†Usually, the only datagrams destined for a router are those used to test connectivity or those that carry router management commands, but a router must also keep a copy of datagrams that are broadcast on the network.

The answer is that hosts not designated to be routers should *not* route datagrams that they receive; they should discard them.

There are four reasons why a host not designated to serve as a router should refrain from performing any router functions. First, when such a host receives a datagram intended for some other machine, something has gone wrong with internet addressing, routing, or delivery. The problem may not be revealed if the host takes corrective action by routing the datagram. Second, routing will cause unnecessary network traffic (and may steal CPU time from legitimate uses of the host). Third, simple errors can cause chaos. Suppose that every host routes traffic, and imagine what happens if one machine accidentally broadcasts a datagram that is destined for some host, *H*. Because it has been broadcast, every host on the network receives a copy of the datagram. Every host forwards its copy to *H*, which will be bombarded with many copies. Fourth, as later chapters show, routers do more than merely route traffic. As the next chapter explains, routers use a special protocol to report errors, while hosts do not (again, to avoid having multiple error reports bombard a source). Routers also propagate routing information to ensure that their routing tables are consistent. If hosts route datagrams without participating fully in all router functions, unexpected anomalies can arise.

8.11 Establishing Routing Tables

We have discussed how IP routes datagrams based on the contents of routing tables, without saying how systems initialize their routing tables or update them as the network changes. Later chapters deal with these questions and discuss protocols that allow routers to keep routes consistent. For now, it is only important to understand that IP software uses the routing table whenever it decides how to forward a datagram, so changing routing tables will change the paths datagrams follow.

8.12 Summary

IP uses routing information to forward datagrams; the computation consists of deciding where to send a datagram based on its destination IP address. Direct delivery is possible if the destination machine lies on a network to which the sending machine attaches; we think of this as the final step in datagram transmission. If the sender cannot reach the destination directly, the sender must forward the datagram to a router. The general paradigm is that hosts send indirectly routed datagrams to the nearest router; the datagrams travel through the internet from router to router until they can be delivered directly across one physical network.

When IP software looks up a route, the algorithm produces the IP address of the next machine (i.e., the address of the next hop) to which the datagram should be sent; IP passes the datagram and next hop address to network interface software. Transmission of a datagram from one machine to the next always involves encapsulating the datagram in a physical frame, mapping the next hop internet address to a physical address, and sending the frame using the underlying hardware.

The internet routing algorithm is table driven and uses only IP addresses. Although it is possible for a routing table to contain a host-specific destination address, most routing tables contain only network addresses, keeping routing tables small. Using a default route can also help keep a routing table small, especially for hosts that can access only one router.

FOR FURTHER STUDY

Routing is an important topic. Frank and Chou [1971] and Schwartz and Stern [1980] discuss routing in general; Postel [1980] discusses internet routing. Braden and Postel [RFC 1009] provides a summary of how Internet routers handle IP datagrams. Narten [1989] contains a survey of Internet routing. Fultz and Kleinrock [1971] analyzes adaptive routing schemes; and McQuillan, Richer, and Rosen [1980] describes the ARPANET adaptive routing algorithm.

The idea of using policy statements to formulate rules about routing has been considered often. Leiner [RFC 1124] considers policies for interconnected networks. Braun [RFC 1104] discusses models of policy routing for internets, Rekhter [RFC 1092] relates policy routing to the second NSFNET backbone, and Clark [RFC 1102] describes using policy routing with IP.

EXERCISES

- 8.1 Complete routing tables for all routers in Figure 8.2. Which routers will benefit most from using a default route?
- 8.2 Examine the routing algorithm used on your local system. Are all the cases mentioned in the chapter covered? Does the algorithm allow anything not mentioned?
- 8.3 What does a router do with the *time to live* value in an IP header?
- 8.4 Consider a machine with two physical network connections and two IP addresses I_1 and I_2 . Is it possible for that machine to receive a datagram destined for I_2 over the network with address I_1 ? Explain.
- 8.5 Consider two hosts, A and B , that both attach to a common physical network, N . Is it ever possible, when using our routing algorithm, for A to receive a datagram destined for B ? Explain.
- 8.6 Modify the routing algorithm to accommodate the IP source route options discussed in Chapter 7.
- 8.7 An IP router must perform a computation that takes time proportional to the length of the datagram header each time it processes a datagram. Explain.
- 8.8 A network administrator argues that to make monitoring and debugging his local network easier, he wants to rewrite the routing algorithm so it tests host-specific routes *before* it tests for direct delivery. How can he use the revised algorithm to build a network monitor?

- 8.9** Is it possible to address a datagram to a router's IP address? Does it make sense to do so?
- 8.10** Consider a modified routing algorithm that examines host-specific routes before testing for delivery on directly connected networks. Under what circumstances might such an algorithm be desirable? undesirable?
- 8.11** Play detective: after monitoring IP traffic on a local area network for 10 minutes one evening, someone notices that all frames destined for machine *A* carry IP datagrams that have destination equal to *A*'s IP address, while all frames destined for machine *B* carry IP datagrams with destination *not* equal to *B*'s IP address. Users report that both *A* and *B* can communicate. Explain.
- 8.12** How could you change the IP datagram format to support high-speed packet switching at routers? Hint: a router must recompute a header checksum after decrementing the time-to-live field.
- 8.13** Compare CLNP, the ISO connectionless delivery protocol (ISO standard 8473) with IP. How well will the ISO protocol support high-speed switching? Hint: variable length fields are expensive.

Internet Protocol: Error And Control Messages (ICMP)

9.1 Introduction

The previous chapter shows how the Internet Protocol software provides an unreliable, connectionless datagram delivery service by arranging for each router to forward datagrams. A datagram travels from router to router until it reaches one that can deliver the datagram directly to its final destination. If a router cannot route or deliver a datagram, or if the router detects an unusual condition that affects its ability to forward the datagram (e.g., network congestion), the router needs to inform the original source to take action to avoid or correct the problem. This chapter discusses a mechanism that internet routers and hosts use to communicate such control or error information. We will see that routers use the mechanism to report problems and hosts use it to test whether destinations are reachable.

9.2 The Internet Control Message Protocol

In the connectionless system we have described so far, each router operates autonomously, routing or delivering datagrams that arrive without coordinating with the original sender. The system works well if all machines operate correctly and agree on routes. Unfortunately, no large communication system works correctly all the time. Besides failures of communication lines and processors, IP fails to deliver datagrams when the destination machine is temporarily or permanently disconnected from the network, when the time-to-live counter expires, or when intermediate routers become so

congested that they cannot process the incoming traffic. The important difference between having a single network implemented with dedicated hardware and an internet implemented with software is that in the former, the designer can add special hardware to inform attached hosts when problems arise. In an internet, which has no such hardware mechanism, a sender cannot tell whether a delivery failure resulted from a local malfunction or a remote one. Debugging becomes extremely difficult. The IP protocol itself contains nothing to help the sender test connectivity or learn about such failures.

To allow routers in an internet to report errors or provide information about unexpected circumstances, the designers added a special-purpose message mechanism to the TCP/IP protocols. The mechanism, known as the *Internet Control Message Protocol (ICMP)*, is considered a required part of IP and must be included in every IP implementation.

Like all other traffic, ICMP messages travel across the internet in the data portion of IP datagrams. The ultimate destination of an ICMP message is not an application program or user on the destination machine, however, but the Internet Protocol software on that machine. That is, when an ICMP error message arrives, the ICMP software module handles it. Of course, if ICMP determines that a particular higher-level protocol or application program has caused a problem, it will inform the appropriate module. We can summarize:

The Internet Control Message Protocol allows routers to send error or control messages to other routers or hosts; ICMP provides communication between the Internet Protocol software on one machine and the Internet Protocol software on another.

Initially designed to allow routers to report the cause of delivery errors to hosts, ICMP is not restricted to routers. Although guidelines restrict the use of some ICMP messages, an arbitrary machine can send an ICMP message to any other machine. Thus, a host can use ICMP to correspond with a router or another host. The chief advantage of allowing hosts to use ICMP is that it provides a single mechanism used for all control and information messages.

9.3 Error Reporting vs. Error Correction

Technically, ICMP is an *error reporting mechanism*. It provides a way for routers that encounter an error to report the error to the original source. Although the protocol specification outlines intended uses of ICMP and suggests possible actions to take in response to error reports, ICMP does not fully specify the action to be taken for each possible error. In short,

When a datagram causes an error, ICMP can only report the error condition back to the original source of the datagram; the source must relate the error to an individual application program or take other action to correct the problem.

Most errors stem from the original source, but others do not. Because ICMP reports problems to the original source, however, it cannot be used to inform intermediate routers about problems. For example, suppose a datagram follows a path through a sequence of routers, $R_1, R_2, \dots, R_k$. If R_k has incorrect routing information and mistakenly routes the datagram to router R_E , R_E cannot use ICMP to report the error back to router R_k ; ICMP can only send a report back to the original source. Unfortunately, the original source has no responsibility for the problem or control over the misbehaving router. In fact, the source may not be able to determine which router caused the problem.

Why restrict ICMP to communication with the original source? The answer should be clear from our discussion of datagram formats and routing in the previous chapters. A datagram only contains fields that specify the original source and the ultimate destination; it does not contain a complete record of its trip through the internet (except for unusual cases where the record route option is used). Furthermore, because routers can establish and change their own routing tables, there is no global knowledge of routes. Thus, when a datagram reaches a given router, it is impossible to know the path it has taken to arrive there. If the router detects a problem, it cannot know the set of intermediate machines that processed the datagram, so it cannot inform them of the problem. Instead of silently discarding the datagram, the router uses ICMP to inform the original source that a problem has occurred, and trusts that host administrators will cooperate with network administrators to locate and repair the problem.

9.4 ICMP Message Delivery

ICMP messages require two levels of encapsulation as Figure 9.1 shows. Each ICMP message travels across the internet in the data portion of an IP datagram, which itself travels across each physical network in the data portion of a frame. Datagrams carrying ICMP messages are routed exactly like datagrams carrying information for users; there is no additional reliability or priority. Thus, error messages themselves may be lost or discarded. Furthermore, in an already congested network, the error message may cause additional congestion. An exception is made to the error handling procedures if an IP datagram carrying an ICMP message causes an error. The exception, established to avoid the problem of having error messages about error messages, specifies that ICMP messages are not generated for errors that result from datagrams carrying ICMP error messages.

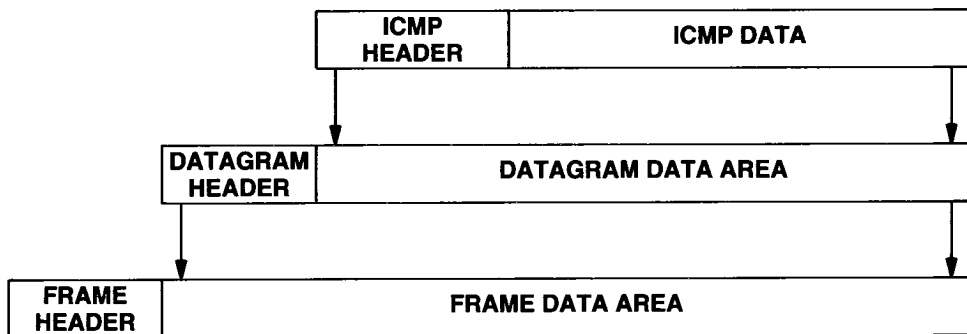


Figure 9.1 Two levels of ICMP encapsulation. The ICMP message is encapsulated in an IP datagram, which is further encapsulated in a frame for transmission. To identify ICMP, the datagram protocol field contains the value *I*.

It is important to keep in mind that even though ICMP messages are encapsulated and sent using IP, ICMP is not considered a higher level protocol — it is a required part of IP. The reason for using IP to deliver ICMP messages is that they may need to travel across several physical networks to reach their final destination. Thus, they cannot be delivered by the physical transport alone.

9.5 ICMP Message Format

Although each ICMP message has its own format, they all begin with the same three fields: an 8-bit integer message *TYPE* field that identifies the message, an 8-bit *CODE* field that provides further information about the message type, and a 16-bit *CHECKSUM* field (ICMP uses the same additive checksum algorithm as IP, but the ICMP checksum only covers the ICMP message). In addition, ICMP messages that report errors always include the header and first 64 data bits of the datagram causing the problem.

The reason for returning more than the datagram header alone is to allow the receiver to determine more precisely which protocol(s) and which application program were responsible for the datagram. As we will see later, higher-level protocols in the TCP/IP suite are designed so that crucial information is encoded in the first 64 bits.

The ICMP *TYPE* field defines the meaning of the message as well as its format. The types include:

Type Field	ICMP Message Type
0	Echo Reply
3	Destination Unreachable
4	Source Quench
5	Redirect (change a route)
8	Echo Request
9	Router Advertisement
10	Router Solicitation
11	Time Exceeded for a Datagram
12	Parameter Problem on a Datagram
13	Timestamp Request
14	Timestamp Reply
15	Information Request (obsolete)
16	Information Reply (obsolete)
17	Address Mask Request
18	Address Mask Reply

The next sections describe each of these messages, giving details of the message format and its meaning.

9.6 Testing Destination Reachability And Status (Ping)

TCP/IP protocols provide facilities to help network managers or users identify network problems. One of the most frequently used debugging tools invokes the ICMP *echo request* and *echo reply* messages. A host or router sends an ICMP echo request message to a specified destination. Any machine that receives an echo request formulates an echo reply and returns it to the original sender. The request contains an optional data area; the reply contains a copy of the data sent in the request. The echo request and associated reply can be used to test whether a destination is reachable and responding. Because both the request and reply travel in IP datagrams, successful receipt of a reply verifies that major pieces of the transport system work. First, IP software on the source computer must route the datagram. Second, intermediate routers between the source and destination must be operating and must route the datagram correctly. Third, the destination machine must be running (at least it must respond to interrupts), and both ICMP and IP software must be working. Finally, all routers along the return path must have correct routes.

On many systems, the command users invoke to send ICMP echo requests is named *ping*[†]. Sophisticated versions of ping send a series of ICMP echo requests, capture responses, and provide statistics about datagram loss. They allow the user to specify the length of the data being sent and the interval between requests. Less sophisticated versions merely send one ICMP echo request and await a reply.

[†]Dave Mills once suggested that *PING* is an acronym for *Packet InterNet Groper*.

9.7 Echo Request And Reply Message Format

Figure 9.2 shows the format of echo request and reply messages.

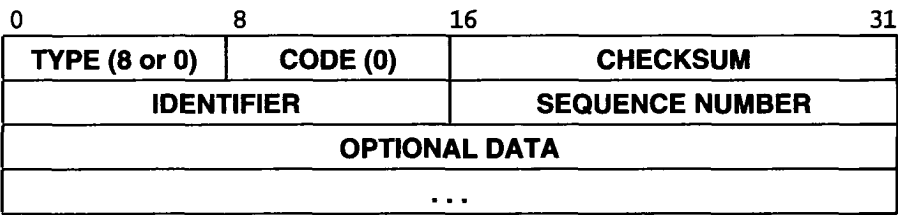


Figure 9.2 ICMP echo request or reply message format.

The field listed as *OPTIONAL DATA* is a variable length field that contains data to be returned to the sender. An echo reply always returns exactly the same data as was received in the request. Fields *IDENTIFIER* and *SEQUENCE NUMBER* are used by the sender to match replies to requests. The value of the *TYPE* field specifies whether the message is a request (8) or a reply (0).

9.8 Reports Of Unreachable Destinations

When a router cannot forward or deliver an IP datagram, it sends a *destination unreachable* message back to the original source, using the format shown in Figure 9.3.

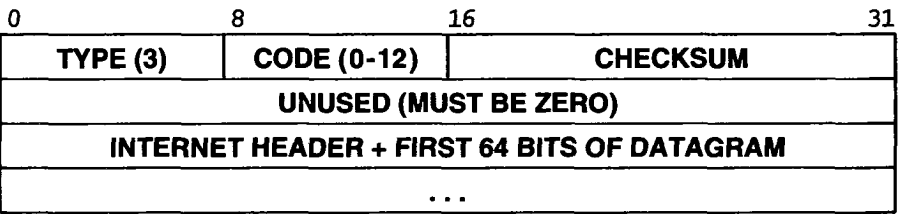


Figure 9.3 ICMP destination unreachable message format.

The *CODE* field in a destination unreachable message contains an integer that further describes the problem. Possible values are:

Code Value	Meaning
0	Network unreachable
1	Host unreachable
2	Protocol unreachable
3	Port unreachable
4	Fragmentation needed and DF set
5	Source route failed
6	Destination network unknown
7	Destination host unknown
8	Source host isolated
9	Communication with destination network administratively prohibited
10	Communication with destination host administratively prohibited
11	Network unreachable for type of service
12	Host unreachable for type of service

Although IP is a best-effort delivery mechanism, discarding datagrams should not be taken lightly. Whenever an error prevents a router from routing or delivering a datagram, the router sends a destination unreachable message back to the source and then *drops* (i.e., discards) the datagram. Network unreachable errors usually imply routing failures; host unreachable errors imply delivery failures[†]. Because the ICMP error message contains a short prefix of the datagram that caused the problem, the source will know exactly which address is unreachable.

Destinations may be unreachable because hardware is temporarily out of service, because the sender specified a nonexistent destination address, or (in rare circumstances) because the router does not have a route to the destination network. Note that although routers report failures they encounter, they may not know of all delivery failures. For example, if the destination machine connects to an Ethernet network, the network hardware does not provide acknowledgements. Therefore, a router can continue to send packets to a destination after the destination is powered down without receiving any indication that the packets are not being delivered. To summarize:

Although a router sends a destination unreachable message when it encounters a datagram that cannot be forwarded or delivered, a router cannot detect all such errors.

The meaning of protocol and port unreachable messages will become clear when we study how higher level protocols use abstract destination points called *ports*. Most of the remaining messages are self explanatory. If the datagram contains the source route option with an incorrect route, it may trigger a *source route* failure message. If a router needs to fragment a datagram but the “don’t fragment” bit is set, the router sends a *fragmentation needed* message back to the source.

[†]An exception occurs for routers using the subnet addressing scheme of Chapter 10. They report a subnet routing failure with an ICMP host unreachable message.

9.9 Congestion And Datagram Flow Control

Because IP is connectionless, a router cannot reserve memory or communication resources in advance of receiving datagrams. As a result, routers can be overrun with traffic, a condition known as *congestion*. It is important to understand that congestion can arise for two entirely different reasons. First, a high-speed computer may be able to generate traffic faster than a network can transfer it. For example, imagine a supercomputer generating internet traffic. The datagrams may eventually need to cross a slower-speed wide area network (WAN) even though the supercomputer itself attaches to a high-speed local area net. Congestion will occur in the router that attaches the LAN to the WAN because datagrams arrive faster than they can be sent. Second, if many computers simultaneously need to send datagrams through a single router, the router can experience congestion, even though no single source causes the problem.

When datagrams arrive too quickly for a host or router to process, it enqueues them in memory temporarily. If the datagrams are part of a small burst, such buffering solves the problem. If the traffic continues, the host or router eventually exhausts memory and must discard additional datagrams that arrive. A machine uses ICMP *source quench* messages to report congestion to the original source. A source quench message is a request for the source to reduce its current rate of datagram transmission. Usually, congested routers send one source quench message for every datagram that they discard. Routers may also use more sophisticated congestion control techniques. Some monitor incoming traffic and quench sources that have the highest datagram transmission rates. Others attempt to avoid congestion altogether by arranging to send quench requests as their queues start to become long, but before they overflow.

There is no ICMP message to reverse the effect of a source quench. Instead, a host that receives source quench messages for a destination, *D*, lowers the rate at which it sends datagrams to *D* until it stops receiving source quench messages; it then gradually increases the rate as long as no further source quench requests are received.

9.10 Source Quench Format

In addition to the usual ICMP *TYPE*, *CODE*, *CHECKSUM* fields, and an unused 32-bit field, source quench messages have a field that contains a datagram prefix. Figure 9.4 illustrates the format. As with most ICMP messages that report an error, the datagram prefix field contains a prefix of the datagram that triggered the source quench request.

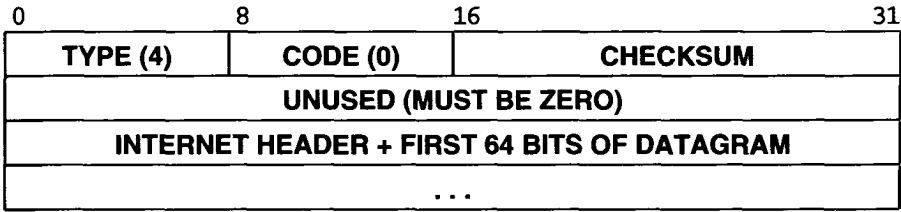


Figure 9.4 ICMP source quench message format. A congested router sends one source quench message each time it discards a datagram; the datagram prefix identifies the datagram that was dropped.

9.11 Route Change Requests From Routers

Internet routing tables usually remain static over long periods of time. Hosts initialize them from a configuration file at system startup, and system administrators seldom make routing changes during normal operations. If the network topology changes, routing tables in a router or host may become incorrect. A change can be temporary (e.g., when hardware needs to be repaired) or permanent (e.g., when a new network is added to the internet). As we will see in later chapters, routers exchange routing information periodically to accommodate network changes and keep their routes up-to-date. Thus, as a general rule:

Routers are assumed to know correct routes; hosts begin with minimal routing information and learn new routes from routers.

To help follow this rule and to avoid duplicating routing information in the configuration file on each host, the initial host route configuration specifies the minimum possible routing information needed to communicate (e.g., the address of a single router). Thus, the host begins with minimal information and relies on routers to update its routing table. In one special case, when a router detects a host using a nonoptimal route, it sends the host an ICMP message, called a *redirect*, requesting that the host change its route. The router also forwards the original datagram on to its destination.

The advantage of the ICMP redirect scheme is simplicity: it allows a host to boot knowing the address of only one router on the local network. The initial router returns ICMP redirect messages whenever a host sends a datagram for which there is a better route. The host routing table remains small but still contains optimal routes for all destinations in use.

Redirect messages do not solve the problem of propagating routes in a general way, however, because they are limited to interactions between a router and a host on a directly connected network. Figure 9.5 illustrates the limitation. In the figure, assume source *S* sends a datagram to destination *D*. Assume that router *R₁* incorrectly routes the datagram through router *R₂* instead of through router *R₄* (i.e., *R₁* incorrectly chooses

a longer path than necessary). When router R_3 receives the datagram, it cannot send an ICMP redirect message to R_1 because it does not know R_1 's address. Later chapters explore the problem of how to propagate routes across multiple networks.

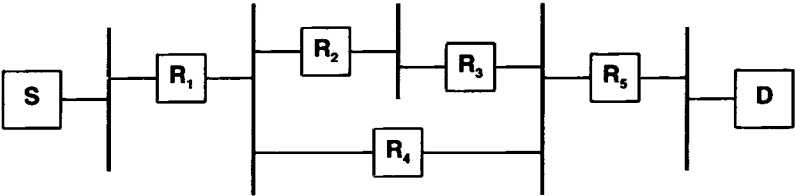


Figure 9.5 ICMP redirect messages do not provide routing changes among routers. In this example, router R_3 cannot redirect R_1 to use the shorter path for datagrams from S to D .

In addition to the requisite *TYPE*, *CODE*, and *CHECKSUM* fields, each redirect message contains a 32-bit *ROUTER INTERNET ADDRESS* field and an *INTERNET HEADER* field, as Figure 9.6 shows.

0	8	16	31
TYPE (5)	CODE (0 to 3)	CHECKSUM	
ROUTER INTERNET ADDRESS			
INTERNET HEADER + FIRST 64 BITS OF DATAGRAM			
...			

Figure 9.6 ICMP redirect message format.

The *ROUTER INTERNET ADDRESS* field contains the address of a router that the host is to use to reach the destination mentioned in the datagram header. The *INTERNET HEADER* field contains the IP header plus the next 64 bits of the datagram that triggered the message. Thus, a host receiving an ICMP redirect examines the datagram prefix to determine the datagram's destination address. The *CODE* field of an ICMP redirect message further specifies how to interpret the destination address, based on values assigned as follows:

Code Value	Meaning
0	Redirect datagrams for the Net (now obsolete)
1	Redirect datagrams for the Host
2	Redirect datagrams for the Type of Service† and Net
3	Redirect datagrams for the Type of Service and Host

As a general rule, routers only send ICMP redirect requests to hosts and not to other routers. We will see in later chapters that routers use other protocols to exchange routing information.

9.12 Detecting Circular Or Excessively Long Routes

Because internet routers compute a next hop using local tables, errors in routing tables can produce a *routing cycle* for some destination, *D*. A routing cycle can consist of two routers that each route a datagram for destination *D* to the other, or it can consist of several routers. When several routers form a cycle, they each route a datagram for destination *D* to the next router in the cycle. If a datagram enters a routing cycle, it will pass around the cycle endlessly. As mentioned previously, to prevent datagrams from circling forever in a TCP/IP internet, each IP datagram contains a time-to-live counter, sometimes called a *hop count*. A router decrements the time-to-live counter whenever it processes the datagram and discards the datagram when the count reaches zero.

Whenever a router discards a datagram because its hop count has reached zero or because a timeout occurred while waiting for fragments of a datagram, it sends an ICMP *time exceeded* message back to the datagram's source, using the format shown in Figure 9.7.

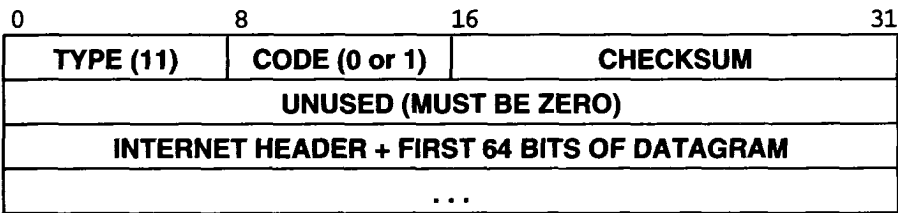


Figure 9.7 ICMP time exceeded message format. A router sends this message whenever a datagram is discarded because the time-to-live field in the datagram header has reached zero or because its reassembly timer expired while waiting for fragments.

ICMP uses the *CODE* field in each time exceeded message (value zero or one) to explain the nature of the timeout being reported:

†Recall that each IP header specifies a type of service used for routing.

Code Value	Meaning
0	Time-to-live count exceeded
1	Fragment reassembly time exceeded

Fragment reassembly refers to the task of collecting all the fragments from a datagram. When the first fragment of a datagram arrives, the receiving host starts a timer and considers it an error if the timer expires before all the pieces of the datagram arrive. Code value 1 is used to report such errors to the sender; one message is sent for each such error.

9.13 Reporting Other Problems

When a router or host finds problems with a datagram not covered by previous ICMP error messages (e.g., an incorrect datagram header), it sends a *parameter problem* message to the original source. One possible cause of such problems occurs when arguments to an option are incorrect. The message, formatted as shown in Figure 9.8, is only sent when the problem is so severe that the datagram must be discarded.

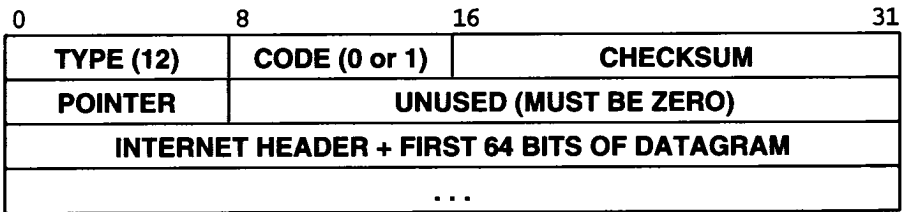


Figure 9.8 ICMP parameter problem message format. Such messages are only sent when the problem causes the datagram to be dropped.

To make the message unambiguous, the sender uses the *POINTER* field in the message header to identify the octet in the datagram that caused the problem. Code 1 is used to report that a required option is missing (e.g., a security option in the military community); the *POINTER* field is not used for code 1.

9.14 Clock Synchronization And Transit Time Estimation

Although machines on an internet can communicate, they usually operate independently, with each machine maintaining its own notion of the current time. Clocks that differ widely can confuse users of distributed systems software. The TCP/IP protocol suite includes several protocols that can be used to synchronize clocks. One of the simplest techniques uses an ICMP message to obtain the time from another machine. A re-

questing machine sends an ICMP *timestamp request* message to another machine, asking that the second machine return its current value for the time of day. The receiving machine returns a *timestamp reply* back to the machine making the request. Figure 9.9 shows the format of timestamp request and reply messages.

0		8	16	31
TYPE (13 or 14)		CODE (0)	CHECKSUM	
IDENTIFIER			SEQUENCE NUMBER	
ORIGINATE TIMESTAMP				
RECEIVE TIMESTAMP				
TRANSMIT TIMESTAMP				

Figure 9.9 ICMP timestamp request or reply message format.

The *TYPE* field identifies the message as a request (13) or a reply (14); the *IDENTIFIER* and *SEQUENCE NUMBER* fields are used by the source to associate replies with requests. Remaining fields specify times, given in milliseconds since midnight, Universal Time†. The *ORIGINATE TIMESTAMP* field is filled in by the original sender just before the packet is transmitted, the *RECEIVE TIMESTAMP* field is filled immediately upon receipt of a request, and the *TRANSMIT TIMESTAMP* field is filled immediately before the reply is transmitted.

Hosts use the three timestamp fields to compute estimates of the delay time between them and to synchronize their clocks. Because the reply includes the *ORIGINATE TIMESTAMP* field, a host can compute the total time required for a request to travel to a destination, be transformed into a reply, and return. Because the reply carries both the time at which the request entered the remote machine, as well as the time at which the reply left, the host can compute the network transit time, and from that, estimate the differences in remote and local clocks.

In practice, accurate estimation of round-trip delay can be difficult and substantially restricts the utility of ICMP timestamp messages. Of course, to obtain an accurate estimate of round trip delay, one must take many measurements and average them. However, the round-trip delay between a pair of machines that connect to a large internet can vary dramatically, even over short periods of time. Furthermore, recall that because IP is a best-effort technology, datagrams can be dropped, delayed, or delivered out of order. Thus, merely taking many measurements may not guarantee consistency; sophisticated statistical analysis is needed to produce precise estimates.

† Universal Time was formerly called Greenwich Mean Time; it is the time of day at the prime meridian.

9.15 Information Request And Reply Messages

The ICMP *information request* and *information reply* messages (types 15 and 16) are now considered obsolete and should not be used. They were originally intended to allow hosts to discover their internet address at system startup. The current protocols for address determination are RARP, described in Chapter 6, and BOOTP, described in Chapter 23.

9.16 Obtaining A Subnet Mask

Chapter 10 discusses the motivation for subnet addressing as well as the details of how subnets operate. For now, it is only important to understand that when hosts use subnet addressing, some bits in the hostid portion of their IP address identify a physical network. To participate in subnet addressing, a host needs to know which bits of the 32-bit internet address correspond to the physical network and which correspond to host identifiers. The information needed to interpret the address is represented in a 32-bit quantity called the *subnet mask*.

To learn the subnet mask used for the local network, a machine can send an *address mask request* message to a router and receive an *address mask reply*. The machine making the request can either send the message directly, if it knows the router's address, or broadcast the message if it does not. Figure 9.10 shows the format of address mask messages.

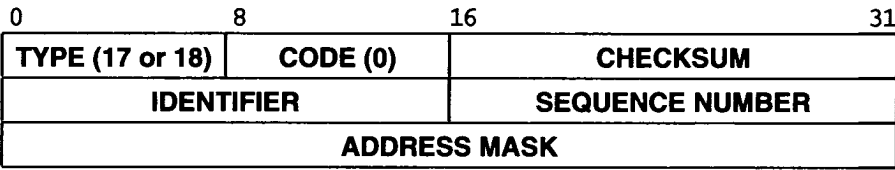


Figure 9.10 ICMP address mask request or reply message format. Usually, hosts broadcast a request without knowing which specific router will respond.

The *TYPE* field in an address mask message specifies whether the message is a request (17) or a reply (18). A reply contains the network's subnet address mask in the *ADDRESS MASK* field. As usual, the *IDENTIFIER* and *SEQUENCE NUMBER* fields allow a machine to associate replies with requests.

9.17 Router Discovery

After a host boots, it must learn the address of at least one router on the local network before it can send datagrams to destinations on other networks. ICMP supports a *router discovery* scheme that allows a host to discover a router address.

ICMP router discovery is not the only mechanism a host can use to find a router address. The BOOTP and DHCP protocols described in Chapter 23 provide the main alternative — each of the protocols provides a way for a host to obtain the address of a default router along with other bootstrap information. However, BOOTP and DHCP have a serious deficiency: the information they return comes from a database that network administrators configure manually. Thus, the information cannot change quickly.

Of course, static router configuration does work well in some situations. For example, consider a network that has only a single router connecting it to the rest of the Internet. There is no need for a host on such a network to dynamically discover routers or change routes. However, if a network has multiple routers connecting it to the rest of the Internet, a host that obtains a default route at startup can lose connectivity if a single router crashes. More important, the host cannot detect the crash.

The ICMP router discovery scheme helps in two ways. First, instead of providing a statically configured router address via a bootstrap protocol, the scheme allows a host to obtain information directly from the router itself. Second, the mechanism uses a *soft state* technique with timers to prevent hosts from retaining a route after a router crashes — routers advertise their information periodically, and a host discards a route if the timer for a route expires.

Figure 9.11 illustrates the format of the advertisement message a router sends.

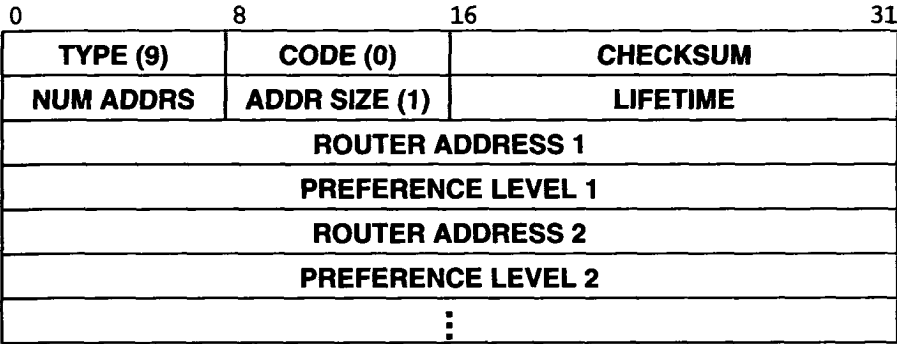


Figure 9.11 ICMP router advertisement message format used with IPv4.
Routers send these messages periodically.

Besides the *TYPE*, *CODE*, and *CHECKSUM* fields, the message contains a field labeled *NUM ADDRS* that specifies the number of address entries which follow (often 1), an *ADDR SIZE* field that specifies the size of an address in 32-bit units (1 for IPv4

addresses), and a *LIFETIME* field that specifies the time in seconds a host may use the advertised address(es). The default value for *LIFETIME* is 30 minutes, and the default value for periodic retransmission is 10 minutes, which means that a host will not discard a route if the host misses a single advertisement message.

The remainder of the message consists of *NUM ADDRS* pairs of fields, where each pair contains a *ROUTER ADDRESS* and an integer *PRECEDENCE LEVEL* for the route. The precedence value is a two's complement integer; a host chooses the route with highest precedence.

If the router and the network support multicast as described in Chapter 17, a router multicasts ICMP router advertisement messages to the all-systems multicast address (i.e., 224.0.0.1). If not, the router sends the messages to the limited broadcast address (i.e., the all 1's address). Of course, a host must never send a router advertisement message.

9.18 Router Solicitation

Although the designers provided a range of values to be used as the delay between successive router advertisements, they chose the default of 10 minutes. The value was selected as a compromise between rapid failure detection and low overhead. A smaller value would allow more rapid detection of router failure, but would increase network traffic; a larger value would decrease traffic, but would delay failure detection. One of the issues the designers considered was how to accommodate a large number of routers on the same network.

From the point of view of a host, the default delay has a severe disadvantage: a host cannot afford to wait many minutes for an advertisement when it first boots. To avoid such delays, the designers included an *ICMP router solicitation message* that allows a host to request an immediate advertisement. Figure 9.12 illustrates the message format.

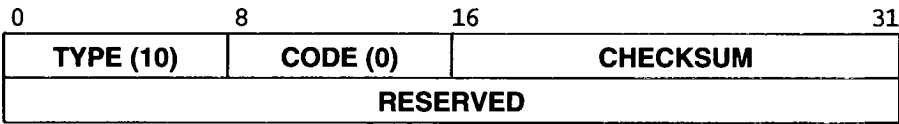


Figure 9.12 ICMP router solicitation message. A host sends a solicitation after booting to request that routers on the local net immediately respond with an ICMP router advertisement.

If a host supports multicasting, the host sends the solicitation to the all-routers multicast address (i.e., 224.0.0.2); otherwise the host sends the solicitation to the limited broadcast address (i.e., the all 1's address). The arrival of a solicitation message causes a router to send a normal router advertisement. As the figure shows, the solicitation does not need to carry information beyond the *TYPE*, *CODE*, and *CHECKSUM* fields.

9.19 Summary

Normal communication across an internet involves sending messages from an application on one host to an application on another host. Routers may need to communicate directly with the network software on a particular host to report abnormal conditions or to send the host new routing information.

The Internet Control Message Protocol provides for extranormal communication among routers and hosts; it is an integral, required part of IP. ICMP includes *source quench* messages that retard the rate of transmission, *redirect* messages that request a host to change its routing table, *echo request/reply* messages that hosts can use to determine whether a destination can be reached, and *router solicitation* and *advertisement* messages that hosts use to dynamically maintain a default route. An ICMP message travels in the data area of an IP datagram and has three fixed-length fields at the beginning of the message: an ICMP message *type* field, a *code* field, and an ICMP *checksum* field. The message type determines the format of the rest of the message as well as its meaning.

FOR FURTHER STUDY

Both Tanenbaum [1981] and Stallings [1985] discuss control messages in general and relate them to various network protocols. The central issue is not how to send control messages but when. Grange and Gien [1979], as well as Driver, Hopewell, and Iaquinto [1979], concentrate on a problem for which control messages are essential, namely, flow control. Gerla and Kleinrock [1980] compares flow control strategies analytically. For a discussion of clock synchronization protocols see Mills [RFCs 956, 957, and 1305].

The Internet Control Message Protocol described here is a TCP/IP standard defined by Postel [RFC 792] and updated by Braden [RFC 1122]. Nagle [RFC 896] discusses ICMP source quench messages and shows how routers should use them to handle congestion control. Prue and Postel [RFC 1016] discusses a more recent technique routers use in response to source quench. Nagle [1987] argues that congestion is always a concern in packet switched networks. Mogul and Postel [RFC 950] discusses subnet mask request and reply messages, and Deering [RFC 1256] discusses the solicitation and advertisement messages used in router discovery. Jain, Ramakrishnan and Chiu [1987] considers how routers and transport protocols could cooperate to avoid congestion.

EXERCISES

- 9.1 Devise an experiment to record how many of each ICMP message type appear on your local network during a day.
- 9.2 Experiment to see if you can send packets through a router fast enough to trigger an ICMP source quench message.
- 9.3 Devise an algorithm that synchronizes clocks using ICMP timestamp messages.
- 9.4 See if your local computer system contains a *ping* command. How does the program interface with protocols in the operating system? In particular, does the mechanism allow an arbitrary user to create a *ping* program, or does such a program require special privilege? Explain.
- 9.5 Assume that all routers send ICMP time-exceeded messages, and that your local TCP/IP software will return such messages to an application program. Use the facility to build a *traceroute* command that reports the list of routers between the source and a particular destination.
- 9.6 If you connect to the global Internet, try to ping host 128.10.2.1 (a machine at Purdue).
- 9.7 Should a router give ICMP messages priority over normal traffic? Why or why not?
- 9.8 Consider an Ethernet that has one conventional host, *H*, and 12 routers connected to it. Find a single (slightly illegal) frame carrying an IP packet that, when sent by host *H*, causes *H* to receive exactly 24 packets.
- 9.9 Compare ICMP source quench packets with Jain's 1-bit scheme used in DECNET. Which is a more effective strategy for dealing with congestion? Why?
- 9.10 There is no ICMP message that allows a machine to inform the source that transmission errors are causing datagrams to arrive corrupted. Explain why.
- 9.11 In the previous question, under what circumstances might such a message be useful?
- 9.12 Should ICMP error messages contain a timestamp that specifies when they are sent? Why or why not?
- 9.13 If routers at your site participate in ICMP router discovery, find out how many addresses each router advertises on each interface.
- 9.14 Try to reach a server on a nonexistent host on your local network. Also try to communicate with a nonexistent host on a remote network. In which case do you receive an error message? Why?
- 9.15 Try using *ping* with a network broadcast address. How many computers answer? Read the protocol documents to determine whether answering a broadcast request is required, recommended, not recommended, or prohibited.

Classless And Subnet Address Extensions (CIDR)

10.1 Introduction

Chapter 4 discusses the original Internet addressing scheme and presents the three primary forms of IP addresses. This chapter examines five extensions of the IP address scheme all designed to conserve network prefixes. The chapter considers the motivation for each extension and describes the basic mechanisms used. In particular, it presents the details of the address subnet scheme that is now part of the TCP/IP standards, and the classless address scheme that is an elective standard.

10.2 Review Of Relevant Facts

Chapter 4 discusses addressing in internetworks and presents the fundamentals of the IP address scheme. We said that the 32-bit addresses are carefully assigned to make the IP addresses of all hosts on a given physical network share a common prefix. In the original IP address scheme, designers thought of the common prefix as defining the network portion of an internet address and the remainder as a host portion. The consequence of importance to us is:

In the original IP addressing scheme, each physical network is assigned a unique network address; each host on a network has the network address as a prefix of the host's individual address.